دانشگاه تهران

دانشکدگان فارابی

دانشکده‌ی مهندسی

گروه مهندسی کامپیوتر

طراحی و پیاده‌سازی دستیار هوشمند بالینی (Clinical AI Copilot) مبتنی بر تلفیق پرونده بیمار و دانش پزشکی معتبر

نگارش

عرفان کربلائی

استاد راهنما

دکتر کاظم فولادی قلعه

گزارش پروژه برای دریافت درجه‌ی کارشناسی

در رشته‌ی مهندسی کامپیوتر

ماه و سال تحویل: ۱۵ مهر ۱۴۰۵ (اکتبر ۲۰۲۶ ۱۵)


بسم الله الرحمن الرحيم

صورت‌جلسه‌ی داوری

محل درج فرم رسمی داوری پس از دریافت از دانشگاه

چکیده

این پروژه یک نمونه‌ی دانشگاهی تحت وب برای ثبت و مرور داده‌های بیمار و استفاده از ابزارهای آماده‌ی هوش مصنوعی است. هسته‌ی سمت سرور با FastAPI و PostgreSQL اطلاعات بالینی، اسناد، یادداشت‌های صوتی و گفتگو را به بیمار مشخص مرتبط می‌کند. فایل اولیه در MinIO نگهداری می‌شود و متن استخراج‌شده، تحلیل سند و متن پاک‌سازی‌شده‌ی صوت از اصل منبع جدا می‌مانند. زمینه‌ی بیمار از رکوردهای جاری ساخته می‌شود و تایم‌لاین ترتیب رخدادهای دارای منبع را عرضه می‌کند.

تمرکز گزارش بر مسئولیت‌های Backend و AI عرفان کربلائی است؛ مسئولیت اصلی رابط طبق پروپوزال با ایمان کربلائی و معماری و یکپارچه‌سازی مشترک است. اتصال مدل متن از سه adapter، گفتگوی ماندگار، کنترل شناسه‌ی شواهد، سه مسیر گفتار به متن، احراز هویت و اجازه‌ی پرونده در کد وجود دارند. مسیر فعلی تحلیل ایمنی دارو بر مدل تکیه دارد و از موتور محدود قواعد قدیمی مستقل است؛ این تفاوت با طرح اولیه به‌طور صریح بررسی می‌شود.

نسخه‌ی موجود استخراج ساختاریافته‌ی خودکار مقادیر آزمایش، بازیابی بیرونی مقاله، اتصال عملیاتی بیمارستانی یا ارزیابی بالینی تأییدشده ندارد. در اجرای منتخب تازه، ۵۹ آزمون موفق و دو آزمون به دلیل آماده نبودن MinIO دچار خطای setup شدند. این نتیجه درباره‌ی صحت قراردادهای آزموده‌شده است و به کیفیت پزشکی مدل تعمیم داده نمی‌شود. دستاورد پروژه، مبنایی قابل بررسی برای یکپارچه‌سازی داده و سرویس است؛ کارهای آینده بر امنیت عملیاتی، وضعیت پردازش پایدار، منشأ دقیق و ارزیابی مبتنی بر داده‌ی مرجع متمرکزند.

کلیدواژه‌ها: دستیار هوشمند بالینی، زمینه‌ی بیمار، FastAPI، پردازش سند، گفتار به متن، شواهد پرونده، ایمنی دارو.

فهرست مطالب

فصل اول مقدمه و معرفی پروژه..... 7
۱-۱ مسئله و جایگاه پروژه..... 7
۱-۲ هدف‌ها و مرز نمونه‌ی اولیه..... 7
۱-۳ تفکیک مسئولیت و سهم عرفان..... 8
۱-۴ وضعیت بررسی‌شده و معیار خواندن گزارش..... 9
فصل دوم تعریف مسئله و مبانی طراحی..... 11
۲-۱ گردش کار بالینی و نیازهای عملکردی..... 11
۲-۲ نیازهای کیفی و معیار پذیرش..... 11
۲-۳ قرارداد API و جدایی لایه‌ها..... 12
۲-۴ پایگاه داده و ذخیره‌سازی شیء..... 13
۲-۵ مدل زبانی و زمینه‌ی کنترل‌شده..... 13
۲-۶ معنای ارزیابی فنی و بالینی..... 14
فصل سوم معماری کلان و جریان داده..... 15
۳-۱ اجزای اجرا و مسئولیت آن‌ها..... 15
۳-۲ چرخه‌ی درخواست و تزریق وابستگی..... 15
۳-۳ چرخه‌ی عمر داده و خروجی مشتق..... 16
۳-۴ مرز رابط کاربری و سمت سرور..... 17
۳-۵ مرزهای اعتماد و نقاط شکست..... 17
فصل چهارم پیاده‌سازی سمت سرور و مدل داده..... 19
۴-۱ ساخت برنامه و سازمان مسیرها..... 19
۴-۲ طرح‌واره‌های داده و اعتبارسنجی (Schema)..... 19
۴-۳ بیمار و مدیریت دسترسی هنگام ایجاد..... 20
۴-۴ دارو و داده‌ی بالینی..... 20
۴-۵ روابط و رفتار حذف..... 21
۴-۶ نشست پایگاه داده و مهاجرت‌ها..... 22

۷-۴ خطا و یکپارچگی عملیات 22
فصل پنجم پرونده بیمار زمینه و تایم‌لاین 25
۵-۱ تجمیع داده بدون تولید اطلاعات تازه 25
۵-۲ ترتیب و سقف اطلاعات 25
۵-۳ تفاوت نمای کلی بیمار با زمینه‌ی بالینی 26
۵-۴ تایم‌لاین مشتق از رکورد 26
۵-۵ کیفیت داده و تفسیر نبود اطلاعات 27
۵-۶ آزمون‌های زمینه و مسیر آینده 28
فصل ششم بارگذاری پردازش و تحلیل اسناد 29
۶-۱ قرارداد بارگذاری و ثبت متادیتا 29
۶-۲ کلید شیء و مسئولیت ذخیره‌ساز 29
۶-۳ استخراج متن و وضعیت کار 30
۶-۴ استخراج متن PDF و تشخیص نوری نویسه‌ها (OCR) 30
۶-۵ تحلیل سند با هوش مصنوعی 31
۶-۶ دانلود حذف و نقاط یکپارچه‌سازی 32
۶-۷ آزمون‌های پردازش سند و مسیر توسعه 32
فصل هفتم مدل زبانی و گفتگوی بالینی 33
۷-۱ قرارداد ارائه‌دهنده و پیکربندی 33
۷-۲ درخواست به ارائه‌دهنده‌های OpenRouter و OpenAI-compatible 34
۷-۳ قالب پیام و پاسخ در Gemini 34
۷-۴ تکرار تماس و خطای شبکه 35
۷-۵ ساخت ورودی مدل و محدودیت زمینه 35
۷-۶ گفتگوی بدون ماندگاری 35
۷-۷ گفتگوی ماندگار و ترتیب ثبت پیام‌ها 36
۷-۸ کاتالوگ و رابط گفتگو 36
۷-۹ دامنه‌ی تضمین و ارزیابی لازم 37

فصل هشتم صوت و تبدیل گفتار به متن 39
۸-۱. ثبت صوت و وضعیت‌های مستقل 39
۸-۲. قرارداد STT و انتخاب ارائه‌دهنده 39
۸-۳. پردازش دسته‌ای صوت در Speechmatics 40
۸-۴. آماده‌سازی فایل و تشخیص گفتار با Google web Speech 40
۸-۵. ارسال ورودی صوت به OpenRouter 41
۸-۶. زنجیره‌ی جایگزین و معنای موفقیت 42
۸-۷. فایل موقت و نکته‌ی Windows 42
۸-۸. پاک‌سازی اختیاری و تأیید یادداشت 42
۸-۹. آزمون و سنجش کیفیت 43
فصل نهم ایمنی دارو شواهد و عامل آزمایشی 45
۹-۱. تغییر جریان ایمنی نسبت به طرح اولیه 45
۹-۲. قرارداد تحلیل ساختاریافته 46
۹-۳. حالت ایمنی در چت 46
۹-۴. موتور قواعد محدود 46
۹-۵. شواهد پرونده و شناسه‌ی پایدار 47
۹-۶. تجزیه‌ی پاسخ و نمایش منبع 48
۹-۷. ابزارهای محلی و عامل آزمایشی MCP 48
۹-۸. امنیت و محدودیت آزمایش عامل 49
فصل دهم احراز هویت امنیت و حریم خصوصی 51
۱۰-۱. حساب کاربری و توکن 51
۱۰-۲. اجازه‌ی پرونده و منابع فرزند 51
۱۰-۳. ورود یک‌بارمصرف 52
۱۰-۴. نگهداری توکن در رابط 53
۱۰-۵. نگهداری فایل، داده و گزارش‌های اجرا 53
۱۰-۶. کنترل‌های موجود و کارهای باقی‌مانده 54

فصل یازدهم آزمون یکپارچه‌سازی و استقرار 55
۱۱-۱. ساختار آزمون‌ها 55
۱۱-۲. بازاجرای آزمون‌های منتخب 55
۱۱-۳. نتایج تاریخی و تفسیر آن‌ها 56
۱۱-۴. طرح ارزیابی و معیارهای غایب 56
۱۱-۵. زیرساخت توسعه 57
۱۱-۶. تصویر Frontend و Backend 57
۱۱-۷. توپولوژی استقرار و ترتیب راه‌اندازی 58
۱۱-۸. Nginx و نشانی امضاشده 58
۱۱-۹. رویه‌ی راه‌اندازی و وضعیت ارزیابی استقرار 59
فصل دوازدهم خلاصه نتیجه‌گیری و کارهای آینده 61
۱۲-۱. تطبیق با هدف‌های مصوب 61
۱۲-۲. سهم مهندسی و حدود نتیجه 61
۱۲-۳. اولویت‌های مهندسی 62
۱۲-۴. برنامه‌ی ارزیابی و توسعه‌ی علمی 62
۱۲-۵. نتیجه‌ی نهایی 63
منابع و مراجع 65
پیوست الف. مسیرهای منتخب رابط برنامه‌نویسی (API) 67
پیوست ب. تنظیمات منتخب اجرا 68

فهرست شکل‌ها

شکل ۳-۱ معماری سامانه و مرزهای ذخیره و پردازش 15
شکل ۳-۲ مسیر درخواست از اعتبارسنجی و مجوز تا پاسخ 16
شکل ۴-۱ رابطه‌های اصلی پایگاه داده 21
شکل ۵-۱ ساخت زمینه از رکوردهای مرتبط بیمار 25
شکل ۶-۱ جریان سند از فایل اولیه تا متن و تحلیل اختیاری 29
شکل ۷-۱ توالی گفتگوی ماندگار و حفظ سؤال در شکست مدل 36
شکل ۸-۱ ارائه‌دهنده‌های گفتار و مسیرهای جایگزین 39
شکل ۹-۱ تفکیک تحلیل AI و موتور محدود قواعد از ثبت دارو 45
شکل ۹-۲ اعتبارسنجی شناسه‌ی شواهد و دسترسی به منبع 47
شکل ۱۰-۱ احراز هویت و کنترل انتساب بیمار 52
شکل ۱۱-۱ استقرار Compose توپولوژی تعریف‌شده در 58

فهرست جدول‌ها

جدول ۱- تقسیم مسئولیت مطابق پروپوزال 8
جدول ۲- نیاز و سطح تحقق 12
جدول ۳- زنجیره‌ی واقعی مهاجرت‌ها 22
جدول ۴- منبع و تاریخ رویدادهای تایم‌لاین 27
جدول ۵- قرارداد ارائه‌دهنده‌های متن 33
جدول ۶- نتیجه‌ی اجرای منتخب 56
جدول ۷- تطبیق خروجی با هدف‌های طرح 61
جدول الف-۱. مسیرهای منتخب API 67
جدول ب-۱. متغیرهای منتخب و حدود تفسیر 68

فصل اول مقدمه و معرفی پروژه

۱-۱. مسئله و جایگاه پروژه

پرونده‌ی بیمار مجموعه‌ای از داده‌های هم‌شکل نیست. مشخصات پایه، وضعیت مصرف دارو، حساسیت‌ها، نتیجه‌ی آزمایش، تصویر یک گزارش و یادداشت پزشک هرکدام ساختار و زمان ثبت متفاوتی دارند. مسئله‌ی مهندسی این پروژه فراهم کردن محیطی است که این منابع را به یک بیمار مشخص مرتبط کند و هنگام مرور پرونده، هم اطلاعات اصلی و هم منشأ آن‌ها در دسترس بماند. وجود یک پاسخ روان از مدل زبانی، به‌تنهایی این مسئله را حل نمی‌کند؛ پاسخ باید در کنار داده‌ای دیده شود که پزشک بتواند آن را بررسی کند.

عنوان مصوب پروژه «طراحی و پیاده‌سازی دستیار هوشمند بالینی (Clinical AI Copilot) مبتنی بر تلفیق پرونده بیمار و دانش پزشکی معتبر» است. نسخه‌ی موجود، نمونه‌ای دانشگاهی برای ثبت و سازمان‌دهی پرونده، آماده‌سازی متن اسناد و صوت و تولید پاسخ کمکی با مدل آماده است. بخش نخست این هدف در گردش کار برنامه پیاده شده است؛ اتصال به منابع بیرونی دانش پزشکی، از جمله مقالات و دانش‌نامه‌ی دارویی، هنوز در این گردش کار وجود ندارد.

پروژه در قلمرو مهندسی نرم‌افزار و یکپارچه‌سازی ابزارهای آماده قرار دارد. آموزش مدل پایه، تشخیص خودکار تصاویر پزشکی و تصمیم مستقل درمانی جزء دامنه‌ی پیاده‌سازی نیستند. اطلاعات بیمار به‌صورت رکوردهای ساختاریافته و مستندات نگهداری می‌شود و خروجی هوش مصنوعی در بخش‌هایی جداگانه قرار می‌گیرد. بنابراین موفقیت اصلی را باید در انسجام قراردادهای داده، قابلیت ردیابی، مدیریت شکست و امکان آزمودن گردش کار جست‌وجو کرد، نه در ادعای دقت تشخیصی.

۱-۲. هدف‌ها و مرز نمونه‌ی اولیه

هدف نخست ایجاد هسته‌ای برای مدیریت بیمار و داده‌های وابسته است. هدف دوم تبدیل سند و صوت به متن قابل استفاده، بدون کنار گذاشتن فایل اصلی است. هدف سوم ساخت زمینه‌ی بیمار برای گفتگوی بالینی و نمایش شواهد پرونده همراه پاسخ است. بررسی ایمنی دارو نیز به‌عنوان یک جریان کمکی وجود دارد، اما نسخه‌ی فعلی آن با طرح اولیه تفاوت دارد: مسیر اصلی رابط از تحلیل مدل زبانی استفاده می‌کند و موتور محدود قواعد همچنان در مسیر دیگری باقی مانده است.

نمونه‌ی اولیه یا MVP، مخفف Minimum Viable Product، مجموعه‌ای محدود از گردش کارهای قابل اجرا برای پزشک است. ثبت دارو با اقدام کاربر انجام می‌شود و تحلیل ایمنی پیش‌شرط ذخیره‌ی آن نیست. تبدیل رونوشت صوت به یادداشت بالینی نیز به درخواست صریح کاربر نیاز دارد. به این ترتیب ثبت اطلاعات از دسترس‌پذیری مدل جدا می‌ماند و مسئولیت بازبینی خروجی و تصمیم بالینی با پزشک است.

در مخزن رابط تبادل با سامانه‌ی واقعی بیمارستان، ورودی استاندارد پرونده‌ی الکترونیک سلامت یا پیاده‌سازی FHIR وجود ندارد. پرونده‌ها از طریق API و داده‌ی نمونه ساخته می‌شوند. همچنین معیارهایی مانند کاهش زمان ویزیت یا بهبود پیامد بیمار در این نسخه اندازه‌گیری نشده‌اند. این موارد در پروپوزال به‌عنوان انگیزه یا روش ارزیابی مطرح شده‌اند و نباید به نتیجه‌ی انجام‌شده تبدیل شوند.

۱-۳. تفکیک مسئولیت و سهم عرفان

بخش تخصیص کار پروپوزال، Backend و هسته‌ی هوشمند را به عرفان واگذار کرده است. این حوزه شامل طراحی API، پایگاه داده، مدیریت داده‌ی بیمار، پردازش سمت سرور سند و صوت، اتصال سرویس تشخیص گفتار، ساخت زمینه، اتصال مدل آماده، ارجاع و ایمنی نسخه است. تمرکز این گزارش بر همین حوزه و تصمیم‌های قابل مشاهده در کد قرار دارد. شناسه‌ی توابع و فایل‌ها در فصل‌های فنی آمده‌اند؛ پیوست الف مسیرهای منتخب و پیوست ب تنظیمات مهم اجرا را خلاصه می‌کند.

مسئولیت اصلی رابط کاربری و تجربه‌ی کاربر در پروپوزال با ایمان کربلائی است: داشبورد، نمای بیمار، بارگذاری، نمایش متن، تایم‌لاین، گفتگو و نمایش هشدار و منبع. معماری کلان، قرارداد API، یکپارچه‌سازی، آزمون سرتاسری، استقرار و نمایش نهایی وظایف مشترک‌اند. در این گزارش، جزئیات رابط تا جایی شرح داده می‌شوند که مسیر داده و اتصال آن به قابلیت‌های سمت سرور روشن شود.

تاریخچه‌ی Git تغییرات مهم Backend را با نویسنده‌ی ErfanKarbalaei ثبت کرده است؛ از جمله مدل داده در 09a4e6d، زمینه‌ی بیمار در 12045ee، گفتگوی ماندگار در 7fc9c53 و شواهد در .7148efd همین نام در ثبت برخی تغییرات Frontend هم دیده می‌شود. نویسنده‌ی یک commit لزوماً نشان‌دهنده‌ی مالکیت انحصاری تمام کار گروهی نیست؛ بنابراین تخصیص رسمی پروپوزال مبنای بیان مسئولیت است و تاریخچه فقط قرینه‌ی فنی برای مسیر تغییرات محسوب می‌شود.

جدول ۱ تقسیم مسئولیت مطابق پروپوزال

حوزه	مسئول اصلی	مرز مسئولیت
Backend و AI	عرفان کربلائی	API و داده، سند و صوت، Context، اتصال مدل، شواهد و ایمنی
Frontend و UX	ایمان کربلائی	رابط بالینی، نمایش پرونده و خروجی، ضبط و بارگذاری و اتصال UI
معماری و تحویل	هر دو دانشجو	قرارداد API، Integration، E2E، Deployment و Demo

۱-۴. وضعیت بررسی‌شده و معیار خواندن گزارش

مبنای فنی گزارش نسخه‌ی کاری مخزن در زمان بررسی است. آخرین commit مشاهده‌شده 56ed451 است و تغییرات ثبت‌نشده‌ی گفتار به متن، تنظیمات و وابستگی‌های متناظر نیز در تحلیل وارد شده‌اند. به همین دلیل مسیرهای Google و OpenRouter در صوت شرح داده می‌شوند، هرچند همه‌ی آن‌ها در آخرین commit ثبت نشده‌اند. فایل‌های برنامه برای تدوین این گزارش تغییر داده نشده‌اند.

وضعیت هر قابلیت با شواهد آن خوانده می‌شود: رفتارهایی مانند اعتبارسنجی ورودی و قالب درخواست ارائه‌دهنده در اجرای منتخب آزموده شده‌اند؛ برخی مسیرهای دارای کد و آزمون، از جمله APIهای وابسته به پایگاه داده، در این اجرا تأیید نشده‌اند؛ عامل محلی MCP آزمایشی است؛ و استخراج ساختاریافته‌ی خودکار و بازیابی دانش پزشکی بیرونی هنوز پیاده نشده‌اند. نتایج اسناد تسک‌ها با تاریخ و دامنه‌ی همان اجرا تفسیر می‌شوند. شرح پیکربندی Docker نیز در فصل استقرار از نتیجه‌ی اجرای عمومی جدا شده است.

فصل‌های بعد از نیازمندی و معماری به پیاده‌سازی داده، زمینه، اسناد، گفتگو، صوت و ایمنی می‌رسند. سپس امنیت، آزمون و استقرار بررسی می‌شوند و فصل پایانی تطبیق هدف‌ها و کارهای آینده را ارائه می‌کند. دو پیوست پایانی فقط مسیرهای منتخب رابط و تنظیمات مهم اجرا را گرد می‌آورند؛ شرح معماری و داده در فصل‌های اصلی آمده است.

فصل دوم تعریف مسئله و مبانی طراحی

۲-۱. گردش کار بالینی و نیازهای عملکردی

گردش کار مورد نظر با یافتن یا ایجاد بیمار آغاز می‌شود. پس از ثبت مشخصات، کاربر مشکل بالینی، حساسیت و دارو را وارد می‌کند و مستندات را به همان پرونده می‌افزاید. برنامه باید میان شناسه‌ی بیمار و شناسه‌ی یک منبع فرزند تمایز بگذارد؛ برای مثال داشتن شماره‌ی یک سند نباید مجوز دیدن آن سند باشد. این نیاز در طراحی مسیرهای تودرتو و در بررسی دسترسی مسیرهای مستقل نقش مستقیم دارد.

نمای کلی بیمار برای مرور اولیه، داروهای فعال، حساسیت‌های شدیدتر و نتایج اخیر را با ترتیب مشخص عرضه می‌کند. تایم‌لاین ترتیب رویدادهای ثبت‌شده و منبع هر رویداد را نشان می‌دهد. پاسخ این دو مسیر با queryهای پایگاه ساخته می‌شود: overview داده‌های منتخب را کنار هم می‌گذارد و timeline از رکوردهای اصلی رویداد می‌سازد. تولید تحلیل روایی با مدل، مسیر جداگانه‌ای دارد که برای اسناد پیاده شده است.

برای سند و صوت، پاسخ اولیه‌ی آپلود و نتیجه‌ی پردازش در دو زمان به دست می‌آیند. اگر کاربر مجبور باشد تا پایان OCR یا رونویسی منتظر پاسخ بارگذاری بماند، خطای شبکه می‌تواند وضعیت ذخیره‌ی فایل را مبهم کند. ثبت متادیتا و سپس اجرای کار پس‌زمینه این دو مرحله را جدا می‌کند. رابط می‌تواند pending یا processing را نشان دهد و بعد وضعیت را دوباره بخواند. با این حال این روش درون فرایند برنامه است و دوام یک صف مستقل را ندارد.

گفتگو باید پرونده‌ی بیمار را در لحظه‌ی پرسش دریافت کند و سؤال قبلی کاربر را نیز در صورت نیاز حفظ کند. این دو نوع زمینه یکسان نیستند: سابقه‌ی گفتگو نشان می‌دهد چه گفته شده، در حالی که پرونده بیان می‌کند چه داده‌ای اکنون ثبت است. تفکیک آن‌ها در سرویس گفتگو جلوی تبدیل تاریخچه‌ی پاسخ مدل به منبع اصلی پرونده را می‌گیرد. با وجود این، پاسخ‌های قدیمی ممکن است به داده‌ای مربوط باشند که بعداً تغییر کرده است؛ مسئله‌ی بازتولید دقیق پاسخ همچنان باقی می‌ماند.

۲-۲. نیازهای کیفی و معیار پذیرش

قابلیت ردیابی در این پروژه یعنی بتوان خروجی مشتق را به رکورد یا فایل اولیه مرتبط کرد. فایل اصلی سند و صوت در ذخیره‌ساز باقی می‌ماند؛ متن استخراج‌شده و متن پاک‌سازی‌شده جای آن را نمی‌گیرند. شناسه‌ی منبع در پاسخ مدل نیز باید از رکورد همان بیمار به دست آید. این تعریف عملی‌تر از ادعای کلی «پاسخ معتبر» است، زیرا یک شناسه‌ی واقعی را می‌توان با کد آزمود، در حالی که اعتبار پزشکی پاسخ به ارزیابی معنایی دیگری نیاز دارد.

آزمون‌پذیری با جدا کردن سرویس‌های خالص از مسیرهای HTTP و با قابل جایگزین بودن ارائه‌دهنده‌ها حاصل می‌شود. موتور قواعد دارویی بدون شبکه کار می‌کند و سرویس STT از یک قرارداد کوچک استفاده

می‌کند. در آزمون می‌توان درخواست خارجی را با پاسخ کنترل‌شده جایگزین کرد. چنین آزمونی صحت ساخت درخواست و مدیریت خطا را بررسی می‌کند، نه کیفیت تشخیص گفتار یا استدلال پزشکی سرویس واقعی را.

تاب‌آوری در اینجا حذف کامل شکست نیست؛ حفظ معنای داده هنگام شکست است. برای مثال در گفتگوی ماندگار، سؤال کاربر ثبت می‌شود، ولی هنگام شکست ارائه‌دهنده هیچ پیام دستیار ساختگی ذخیره نمی‌شود. شکست پاک‌سازی صوت نباید رونویسی موفق را از بین ببرد. شکست تحلیل ایمنی هم نباید به معنی از دست رفتن امکان ثبت دارو باشد. این قراردادهای محدود و روشن، معیار پذیرش مناسب‌تری از عبارت‌هایی مانند «سیستم همیشه کار می‌کند» هستند.

از نظر محرمانگی، حداقل نیاز آن است که احراز هویت و اجازه‌ی پرونده پیش از پردازش داده بررسی شوند. داشتن JWT معتبر، دسترسی به تمام بیماران ایجاد نمی‌کند. داده‌ی ارسالی به مدل بیرونی نیز از نظر طراحی باید حداقل مقدار لازم باشد. کد فعلی بخشی از متن را کوتاه می‌کند، اما ناشناس‌سازی کامل یا حذف مشخصات شناسایی‌کننده را اجرا نمی‌کند. بنابراین کنترل اندازه‌ی ورودی با کنترل حریم خصوصی متفاوت است.

جدول ۲ نیاز و سطح تحقق

نیاز	پیاده‌سازی موجود	وضعیت ارزیابی
مدیریت بیمار	CRUD و انتساب پزشک	کد و آزمون API موجود؛ اجرای DB در این گزارش انجام نشد
مرور پرونده	overview و Context و timeline	تجمیع قطعی؛ بدون ارزیابی زمان مرور
پردازش سند	متن PDF/OCR و ai_summary	استخراج ساختاریافته‌ی فیلد خودکار نیست
مستندسازی صوت	STT و متن خام و یادداشت بازبینی‌شده	providerها موجود؛ WER/CER اندازه‌گیری نشده
گفتگو و ارجاع	پیام ماندگار و evidence واقعی	اعتبار ID کنترل می‌شود؛ صحت معنایی جداست
ایمنی نسخه	AI-first و موتور قدیمی مستقل	فاقد اعتبارسنجی بالینی
دانش بیرونی	در طرح اختیاری/آینده	PubMed و RxNorm و FDA متصل نشده‌اند

۲-۳. قرارداد API و جدایی لایه‌ها

API، مخفف Application Programming Interface، قرارداد ارتباط رابط و Backend است. در این پروژه درخواست‌های JSON برای داده‌ی ساختاریافته و multipart برای فایل استفاده می‌شوند. Pydantic نوع و برخی محدودیت‌های ورودی را تعریف می‌کند؛ مدل SQLAlchemy مسئول نگاشت داده به جدول است.

داشتن دو نمایش جدا باعث می‌شود مثلاً password_hash یا storage_key الزاماً به پاسخ عمومی وارد نشوند و تغییرات پایگاه داده مستقیماً به تمام مصرف‌کنندگان تحمیل نشود.

طراحی مسیرها بر منابع استوار است. ایجاد سند در مسیر وابسته به بیمار انجام می‌شود و خواندن یا حذف یک سند با شناسه‌ی خود آن صورت می‌گیرد. در مسیر اول، وابستگی patient_from_id هم بیمار را پیدا می‌کند و هم مجوز را می‌سنجد. در مسیر دوم، ابتدا منبع خوانده می‌شود و سپس patient_id آن برای بررسی دسترسی استفاده می‌شود. این انتخاب باید در تمام منابع فرزند رعایت شود، زیرا کنترل مسیر فهرست به‌تنهایی جلوی دسترسی مستقیم با شناسه را نمی‌گیرد.

کد وضعیت HTTP بخشی از قرارداد است. ۴۰۱ برای نبودن یا نامعتبر بودن احراز هویت، ۴۰۳ برای نبود اجازه، ۴۰۴ برای منبع پیدا نشده و ۴۲۲ برای ورودی نامعتبر به کار می‌رود. ۵۰۳ می‌تواند نشان دهد سرویس لازم تنظیم نشده یا در دسترس نیست و ۵۰۲ شکست ارائه‌دهنده‌ی خارجی را بیان کند. همه‌ی خطاهای پیش‌فرض چارچوب الزاماً فارسی نیستند؛ مسیرها در موارد مشخص detail فارسی دارند و رابط برای برخی خطاها پیام عمومی می‌سازد.

۲-۴. پایگاه داده و ذخیره‌سازی شیء

PostgreSQL برای داده‌هایی مناسب این معماری است که به رابطه، قید یکتایی و تراکنش نیاز دارند. نام بیمار، انتساب پزشک و ارتباط آزمایش با سند در قالب جدول قابل کنترل‌اند. فایل چندمگابایتی نیاز متفاوتی دارد: باید بایت‌های اولیه‌ی آن نگهداری و مستقل دانلود شوند. MinIO این وظیفه را با رابط سازگار با S3 انجام می‌دهد. مبنای این جداسازی، تفاوت چرخه‌ی عمر داده و فایل است، نه صرفاً انتخاب دو فناوری رایج.

تراکنش پایگاه داده مجموعه‌ای از تغییرات مربوط را با هم ثبت یا بازگردانی می‌کند. ساخت مدخل دارو و نسخه‌ی بیمار می‌تواند در یک تراکنش باشد، اما عملیات MinIO در همان تراکنش قرار ندارد. اگر آپلود شیء موفق و ثبت متادیتا ناموفق شود، احتمال فایل بدون رکورد وجود دارد. اگر حذف فایل موفق و حذف رکورد ناموفق شود، جهت مسئله برعکس می‌شود. نسخه‌ی فعلی هماهنگ‌سازی پایدار میان این دو سامانه را تضمین نمی‌کند و این موضوع یک محدودیت مشخص معماری است.

قید خارجی و رفتار حذف، معنا دارند. CASCADE برای داده‌ی وابسته به بیمار مناسب است؛ SET NULL برای پیوند اختیاری آزمایش به سند اجازه می‌دهد مقدار آزمایش پس از حذف سند باقی بماند. این دو رفتار یکسان نیستند. مستندات رسمی PostgreSQL توضیح قیدهای ارجاعی را ارائه می‌کند [۴]؛ انتخاب‌های خاص این پروژه از مدل‌ها و مهاجرت‌ها برداشت می‌شود.

۲-۵. مدل زبانی و زمینه‌ی کنترل‌شده

مدل زبانی بزرگ یا LLM، مخفف Large Language Model، در این سامانه یک سرویس آماده‌ی تولید متن است. به‌جای آنکه پزشک همه‌ی پرونده را در هر سؤال تایپ کند، Backend زمینه را از داده‌های مرتبط

بیمار می‌سازد. پیام سیستمی حدود نقش را بیان می‌کند و پیام کاربر شامل JSON پرونده و پرسش است. این روش ارتباط پاسخ با داده‌ی بیمار را هدف می‌گیرد، اما تضمین نمی‌کند که مدل همیشه دستورها را رعایت کند.

متکی کردن پاسخ به زمینه یا Grounding در این نسخه با خواندن رکوردهای بیمار و قرار دادن آن‌ها در پیام مدل انجام می‌شود. بازیابی یا Retrieval از مقاله و منبع خارجی به مؤلفه‌های دیگری مانند انتخاب قطعه و رتبه‌بندی نیاز دارد که در این مسیر حاضر نیستند. تولید تقویت‌شده با بازیابی یا RAG، مخفف Retrieval-Augmented Generation، در پروپوزال اختیاری است؛ توسعه‌ی آن باید با توجه به نیاز و ارزش افزوده‌ی قابل اندازه‌گیری انجام شود.

سند و متن صوت می‌توانند علاوه بر اطلاعات بیمار شامل عبارت‌های دستورگونه باشند. پیام سیستمی از مدل می‌خواهد داده را جعل نکند، اما متن ورودی منبع مورد اعتماد برای دستور دادن نیست. کد، جداکننده‌ی قطعی یا پالایش تخصصی حمله‌ی prompt injection ندارد. طراحی آینده باید با پرونده به‌عنوان داده و با دستور سیستم به‌عنوان سیاست برخورد کند و آزمون‌های خصمانه را نیز به آزمون‌های معمول بیفزاید.

۲-۶. معنای ارزیابی فنی و بالینی

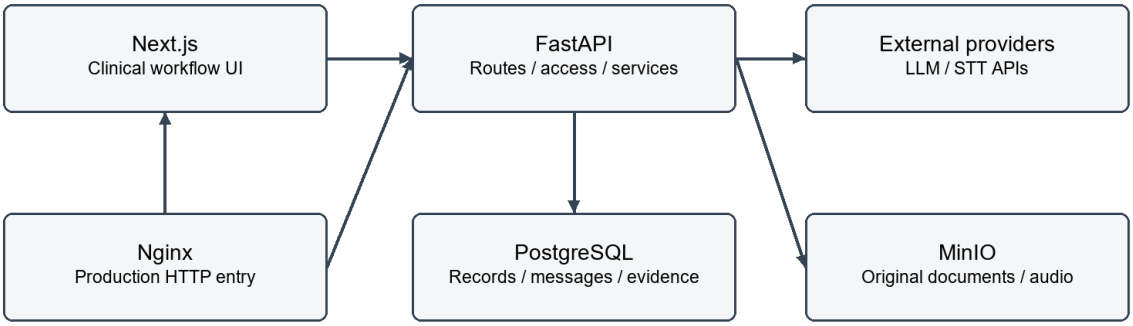
معیارهای پروپوزال از چند جنس‌اند: تطابق فیلد استخراج‌شده، خطای رونویسی، صحت ارجاع، کیفیت پاسخ، حساسیت هشدار، تأخیر و کاربری. نمی‌توان همه‌ی آن‌ها را با شمارش تست‌های API جایگزین کرد. آزمون API که برای ورودی نامعتبر ۴۲۲ می‌بیند، کیفیت پاسخ مدل را نمی‌سنجد. آزمون mocked STT که سه قطعه را به ترتیب کنار هم می‌گذارد، نرخ خطای کلمه روی صدای فارسی را اندازه نمی‌گیرد.

پذیرش مهندسی باید مرحله‌ای باشد: ابتدا صحت ذخیره و دسترسی، سپس صحت قرارداد پردازش، بعد رفتار سرویس واقعی و در پایان ارزیابی انسانی بر داده‌ی مرجع. برای ادعای کاهش خطای دارویی، به سناریوهای مرجع و تحلیل مثبت و منفی کاذب نیاز است. برای ادعای سرعت بهتر ثبت صوت، باید زمان کاربر و زمان تصحیح متن هم اندازه‌گیری شود. گزارش حاضر بخش‌های موجود و روش پیشنهادی برای بخش‌های اندازه‌گیری‌نشده را جدا نگه می‌دارد.

فصل سوم معماری کلان و جریان داده

۳-۱. اجزای اجرا و مسئولیت آن‌ها

معماری شامل رابط Next.js، سمت سرور FastAPI، پایگاه PostgreSQL و ذخیره‌ساز MinIO است. سرویس‌های مدل و تبدیل گفتار به متن بیرون از این هسته قرار دارند و Backend تماس با آن‌ها را هماهنگ می‌کند. Nginx در پیکربندی استقرار نقطه‌ی ورودی مشترک است. شکل ۳-۱ مسیرهای ارتباط تعریف‌شده در کد و پیکربندی را نشان می‌دهد.



شکل ۳-۱ معماری سامانه و مرزهای ذخیره و پردازش

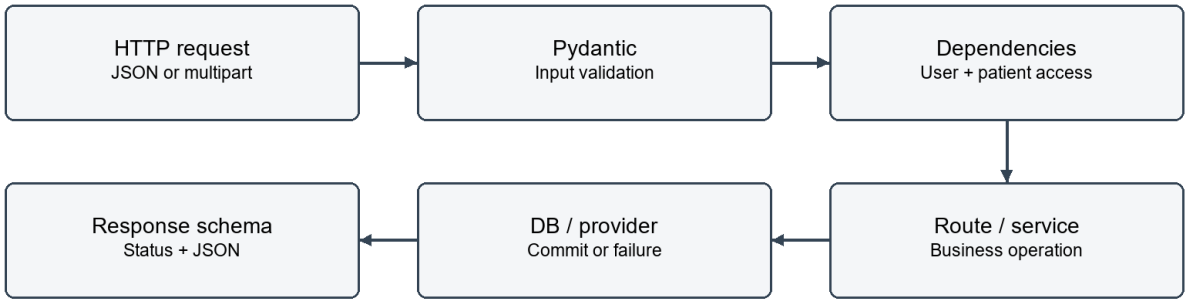
Frontend مسئول نمایش و تعامل کاربر است و ارتباطش با داده از طریق قرارداد API انجام می‌شود. Backend اعتبارسنجی، مجوز، تراکنش و انتخاب ارائه‌دهنده را در اختیار دارد. پایگاه داده منبع اصلی اطلاعات ثبت‌شده و MinIO محل فایل اولیه است. مدل خارجی نقش تولید خروجی مشتق را دارد. با این تفکیک، قطع مدل نباید اصل رکورد بیمار را از بین ببرد و رابط نباید برای دسترسی به پایگاه داده اعتبار مستقلی داشته باشد.

کد سمت سرور به مسیرهای API و وابستگی‌های آن، طرح‌واره‌های داده، مدل‌های رابطه‌ای، سرویس‌های تخصصی و تنظیمات تقسیم شده است. این اجزا در پوشه‌های api، schemas، models، services و core قرار دارند. مرز لایه‌ها کامل نیست: queryهای برخی منابع در route اجرا می‌شوند و عملیات تخصصی مانند تجمیع زمینه و پردازش سند در service قرار دارند. این سازمان‌دهی پیگیری گردش کار را آسان می‌کند، اما تغییر مشترک سیاست query هنوز به بازبینی هر دو محل نیاز دارد.

۳-۲. چرخه‌ی درخواست و تزریق وابستگی

ورود HTTP به برنامه‌ی ساخته‌شده با create_app می‌رسد. مسیرهای api_router زیر پیشوند API_V1_PREFIX ثبت می‌شوند. وابستگی get_db یک Session برای درخواست ایجاد می‌کند و در پایان آن را می‌بندد. get_current_user توکن را می‌خواند و کاربر فعال را بازیابی می‌کند. در مسیر وابسته به بیمار،

patient_from_id علاوه بر خواندن بیمار، انتساب دسترسی را بررسی می‌کند. شکل ۲-۳ جای بررسی مجوز را پیش از ورود به منطق کاربردی نشان می‌دهد.



شکل ۲-۳ مسیر درخواست از اعتبارسنجی و مجوز تا پاسخ

وابستگی‌ها باعث می‌شوند منطق تکراری در تمام مسیرها بازنویسی نشود. همین نقطه‌ی مشترک در آزمون نیز قابل override است. با این حال override کردن کاربر به admin در fixture عمومی، آزمون را برای بررسی رفتار اصلی مسیر ساده می‌کند ولی امنیت واقعی آن مسیر را نمی‌سنجد. مخزن برای این منظور تست‌های جداگانه‌ی احراز هویت و دسترسی دارد. تحلیل نتیجه باید بداند کدام dependency واقعی و کدام جایگزین شده است.

کارهای پس‌زمینه پس از پاسخ اولیه اجرا می‌شوند و Session مستقل می‌سازند؛ Session درخواست در این زمان ممکن است بسته باشد. شناسه‌ی رکورد، به‌جای شیء Session یا فایل باز، به تابع پردازش فرستاده می‌شود. این تصمیم از استفاده‌ی نابجا از منابع درخواست جلوگیری می‌کند. مستندات FastAPI نیز اجرای کار پس از ارسال پاسخ را توضیح می‌دهد [۱]؛ ویژگی دوام و بازگشت پس از crash در کد فعلی به این مکانیزم افزوده نشده است.

۳-۳. چرخه‌ی عمر داده و خروجی مشتق

داده‌ی بیمار از چند مرحله عبور می‌کند: ثبت اولیه، پردازش، ارائه در زمینه و تولید پاسخ. اصل فایل در مرحله‌ی پردازش عوض نمی‌شود. متن استخراج‌شده برای نمایش و مدل قابل استفاده است؛ ai_summary تحلیل ثانویه است. در صوت نیز transcript، cleaned_note و ClinicalNote سه مفهوم متفاوت دارند. تغییر عنوان یا نوع سند، تغییر فایل نیست و تأیید یک یادداشت جدید نیز بازنویسی transcript محسوب نمی‌شود.

داده‌ی مشتق به زمان و ورودی وابسته است. اگر استخراج دوباره اجرا شود، خلاصه‌ی قبلی سند ممکن است بر متن قبلی تکیه داشته باشد؛ مسیر استخراج در کد فعلی به‌طور صریح خلاصه‌ی AI را باطل نمی‌کند. در مقابل پردازش مجدد صوت cleaned_note را پاک و وضعیت پاک‌سازی را pending می‌کند. این تفاوت نشان

می‌دهد سیاست بی‌اعتبار کردن داده‌ی مشتق در تمام زیرسامانه‌ها یکسان نشده است. افزودن نسخه یا fingerprint ورودی می‌تواند این رابطه را قابل کنترل‌تر کند.

در پاسخ چت، زمینه در هر بار ارسال تازه ساخته می‌شود. این کار احتمال پاسخ بر پرونده‌ی قدیمی را کم می‌کند، اما snapshot ورودی ذخیره نمی‌شود. رکورد پیام شامل متن و شواهد است؛ مدل انتخابی، نسخه‌ی prompt و hash زمینه در آن ثبت نمی‌شوند. اگر بعداً بخواهیم علت پاسخ را با همان ورودی بازتولید کنیم، تاریخچه‌ی پیام به‌تنهایی کافی نیست. این محدودیت با ارزش داشتن داده‌ی زنده تعارض ندارد؛ دو نیاز متفاوت‌اند که باید جدا طراحی شوند.

۳-۴. مرز رابط کاربری و سمت سرور

فایل frontend/src/lib/api.ts یک client نوع‌دار دارد. درخواست‌ها cache: no-store دارند و بدنه‌ی FormData بدون تعیین دستی Content-Type ارسال می‌شود تا مرورگر boundary را بسازد. خطای fetch به ApiError با status صفر تبدیل می‌شود. برای ۴۲۲ پیام عمومی اعتبارسنجی و برای بسیاری از وضعیت‌های دیگر detail پاسخ استفاده می‌شود. این لایه رابط را از تکرار قراردادهای شبکه در هر component بی‌نیاز می‌کند.

در مرورگر، token از localStorage خوانده و در Authorization قرار می‌گیرد. در Server Component، cookie access_token به Bearer تبدیل می‌شود. این دو مسیر به یک Backend متکی‌اند، اما خصوصیات امنیتی یکسانی ندارند. cookie توسط JavaScript نوشته شده است و HttpOnly نیست. این جزئیات در فصل امنیت بررسی می‌شود. استفاده‌ی یکسان از URL API برای fetch سمت سرور نیز علت نیاز پیکربندی استقرار به نشانی کامل است.

helper دانلود سند در api.ts فقط نشانی مسیر Backend را برمی‌گرداند. EvidenceList این نشانی را در پیوند عادی مرورگر باز می‌کند؛ چنین پیوندی Bearer ذخیره‌شده در localStorage را اضافه نمی‌کند، در حالی که مسیر دانلود Backend آن را می‌خواهد. بنابراین مسیر API و کلیک رابط از نظر انتقال مجوز یکسان عمل نمی‌کنند. آزمون مرورگر باید این اتصال را بررسی کند؛ راهکارهایی مانند دریافت مجوزدار با fetch یا واسط سروری در بخش کارهای آینده مطرح شده‌اند.

۳-۵. مرزهای اعتماد و نقاط شکست

سه مرز مهم وجود دارد: مرورگر به API، API به ذخیره‌ساز و API به ارائه‌دهنده‌ی خارجی. ورودی مرورگر نیازمند نوع و مجوز است. فایل ذخیره‌شده می‌تواند محتوای خراب یا ناسازگار با MIME اعلامی داشته باشد. پاسخ خارجی نیز ممکن است خطای HTTP، متن ناقص یا JSON نامعتبر باشد. هر مرز کنترل مخصوص خود را می‌خواهد؛ قید پایگاه داده نمی‌تواند صحت JSON مدل را تضمین کند و prompt نمی‌تواند جای ACL را بگیرد.

محدودیت حجم آپلود پس از خواندن کامل فایل اعمال می‌شود. این کنترل جلوی ثبت فایل بزرگ را می‌گیرد، اما مصرف حافظه‌ی اولیه را محدود نمی‌کند. در استقرار Nginx حد ۲۵ مگابایت دارد که برای اسناد ۲۰ مگابایتی مناسب است، ولی از سقف ۵۰ مگابایتی صوت کوچک‌تر است. در نتیجه یک صوت مجاز از نگاه Backend ممکن است پیش از رسیدن به آن رد شود. این اختلاف، مسئله‌ی قرارداد میان لایه‌هاست و باید در برنامه‌ی استقرار حل شود.

سرویس‌های بیرونی با timeout محدود شده‌اند و برخی خطاهای حمل‌ونقل در لایه‌ی AI دوباره امتحان می‌شوند. timeout هر تماس برابر بودجه‌ی کل گردش کار نیست. چند تلاش، انتظار بین تلاش‌ها و چند فراخوانی عامل می‌توانند زمان نهایی را بیشتر کنند. در تحلیل ظرفیت باید طول هر مرحله و تعداد درخواست‌ها ثبت شود؛ هیچ مقدار P95 از این تنظیمات قابل استنتاج نیست.

فصل چهارم پیاده‌سازی سمت سرور و مدل داده

۴-۱. ساخت برنامه و سازمان مسیرها

تابع create_app در backend/app/main.py تنظیمات را دریافت می‌کند، logging را آماده می‌سازد و نمونه‌ی FastAPI را با نام، شرح و نسخه ایجاد می‌کند. سپس CORS و api_router اضافه می‌شوند. مسیر /health خارج از پیشوند نسخه نیز وجود دارد و پاسخ status=ok می‌دهد. این مسیر وجود فرایند پاسخ‌گو را نشان می‌دهد، اما اتصال به PostgreSQL، MinIO یا AI را بررسی نمی‌کند.

فایل api/v1/api.py پانزده router را ثبت می‌کند: سلامت، ورود با رمز و کد یک‌بارمصرف، کاتالوگ مدل، بیمار و منابع بالینی، اسناد، صوت، گفتگو، دو مسیر ایمنی و عامل. این تجمیع، سطح API را در یک محل قابل بررسی می‌کند. برای تعیین رفتار جاری، فراخوانی واقعی تابع‌ها مبناست؛ بعضی commentها همچنان به مراحل قدیمی توسعه اشاره دارند.

میان مسیر خواندن داده و مسیر تحلیل تفاوت وجود دارد. GET اطلاعات ذخیره‌شده را بازمی‌گرداند؛ POST analyze یا ai-safety-check یک فراخوانی خارجی انجام می‌دهد. اجرای مجدد استخراج با ۲۰۲ پذیرفته می‌شود، ولی تحلیل AI سند همگام است. دانستن این تفاوت برای طراحی loading و timeout رابط لازم است و نمی‌توان همه‌ی عملیات پردازش را یک کار غیرهمگام فرض کرد.

۴-۲. طرح‌واره‌های داده و اعتبارسنجی (Schema)

PatientCreate از PatientBase استفاده می‌کند. نام و نام خانوادگی یک تا صد نویسه دارند. کد ملی اختیاری است و در صورت ارسال باید ده رقم باشد؛ این شرط فقط قالب ورودی را کنترل می‌کند و شامل الگوریتم رقم کنترل نیست. تاریخ تولد PastDate، جنسیت Gender و شماره‌ی تماس رشته‌ای با طول محدود است؛ نشانی نیز متن است. این قیدها خطاهای قراردادی ورودی را پیش از رسیدن به منطق ثبت مشخص می‌کنند.

در درخواست‌های PATCH، فیلدها اختیاری‌اند و model_dump(exclude_unset=True) فقط مقادیر ارسال‌شده را اعمال می‌کند. ارسال null با حذف فیلد از درخواست تفاوت دارد: مقادیر ستون‌های nullable قابل پاک شدن‌اند، اما برای ستون غیرnullable ممکن است خطا در مرحله‌ی پایگاه رخ دهد. این تفاوت باید در آزمون طرح‌واره‌های به‌روزرسانی بررسی شود. شرط حداقل طول یک نویسه نیز به‌تنهایی رشته‌ی حاوی فاصله را رد نمی‌کند؛ این کار به strip و اعتبارسنج مناسب نیاز دارد.

پاسخ‌ها با from_attributes از ORM ساخته می‌شوند. این تبدیل محدوده‌ی داده‌ی عمومی را تعریف می‌کند و تاریخ و enum را به نمایش سازگار با JSON می‌برد. وجود relationship بارگذاری‌نشده می‌تواند

هنگام ساخت پاسخ query دیگری ایجاد کند؛ برای داده‌ی دارو در Context از selectinload استفاده شده است. بنابراین schema فقط مسئله‌ی ظاهر JSON نیست و با زمان خواندن داده نیز ارتباط دارد.

۴-۳. بیمار و مدیریت دسترسی هنگام ایجاد

ایجاد بیمار با ساخت Patient و commit انجام می‌شود. IntegrityError به خطای ۴۰۹ با پیام کد ملی تکراری تبدیل می‌شود. برای کاربر doctor، پس از ثبت بیمار یک PatientUserAccess ساخته و دوباره commit می‌شود. ادمین نیاز به انتساب ندارد. این دو commit باعث می‌شود خرابی مرحله‌ی انتساب بتواند بیمار ثبت‌شده‌ای بر جا بگذارد که پزشک سازنده به آن دسترسی ندارد. اتمیک کردن هر دو مرحله، بهبود قابل مشخصی است.

فهرست بیماران q، limit و offset می‌پذیرد. limit پیش‌فرض بیست و حداکثر صد است. جست‌وجو روی نام، نام خانوادگی و کد ملی با ilike انجام می‌شود؛ تعداد کل از subquery شمارش و items با ترتیب id خوانده می‌شوند. برای doctor join با جدول انتساب انجام می‌گیرد. این روش دسترسی فهرست را محدود می‌کند، ولی offset در داده‌ی بزرگ و تغییرات هم‌زمان می‌تواند هزینه و بی‌ثباتی بیشتری از صفحه‌بندی مبتنی بر cursor داشته باشد.

حذف بیمار، db.delete و commit دارد. روابط ORM و FKها داده‌های وابسته‌ی پایگاه را پاک می‌کنند. این cascade فایل‌های MinIO را حذف نمی‌کند، زیرا ذخیره‌ساز بیرون از پایگاه داده است و این route فراخوانی حذف شیء ندارد. بنابراین حذف پرونده در لایه‌ی رابطه‌ای، پاک‌سازی کامل تمام بایت‌های مرتبط محسوب نمی‌شود. سیاست نگهداری و حذف فایل باید مستقل و قابل آزمون طراحی شود.

۴-۴. دارو و داده‌ی بالینی

Medication مدخل مشترک نام و generic_name است و PatientMedication وضعیت مصرف، dosage، frequency، تاریخ شروع و پایان و notes را به بیمار مرتبط می‌کند. این تفکیک اجازه می‌دهد یک نام دارو به چند بیمار متصل شود و تاریخچه‌ی مصرف از هویت کاتالوگ جدا بماند. در نسخه‌ی موجود نام، جای شناسه‌ی استاندارد دارویی را نگرفته است؛ RxCUI یا اتصال RxNorm در مدل نیست.

get_or_create_medication نام را strip می‌کند و با lower در پایگاه جست‌وجو می‌کند. در صورت نبود مدخل، flush انجام می‌دهد و commit را به caller واگذار می‌کند تا همراه نسخه‌ی بیمار ثبت شود. یکتایی ستون name در پایگاه با جست‌وجوی case-insensitive کاملاً یکسان نیست؛ همچنین دو درخواست هم‌زمان می‌توانند هر دو نبود رکورد را ببینند. برای محیط پرترافیک باید قید سازگار و رسیدگی به race بررسی شود.

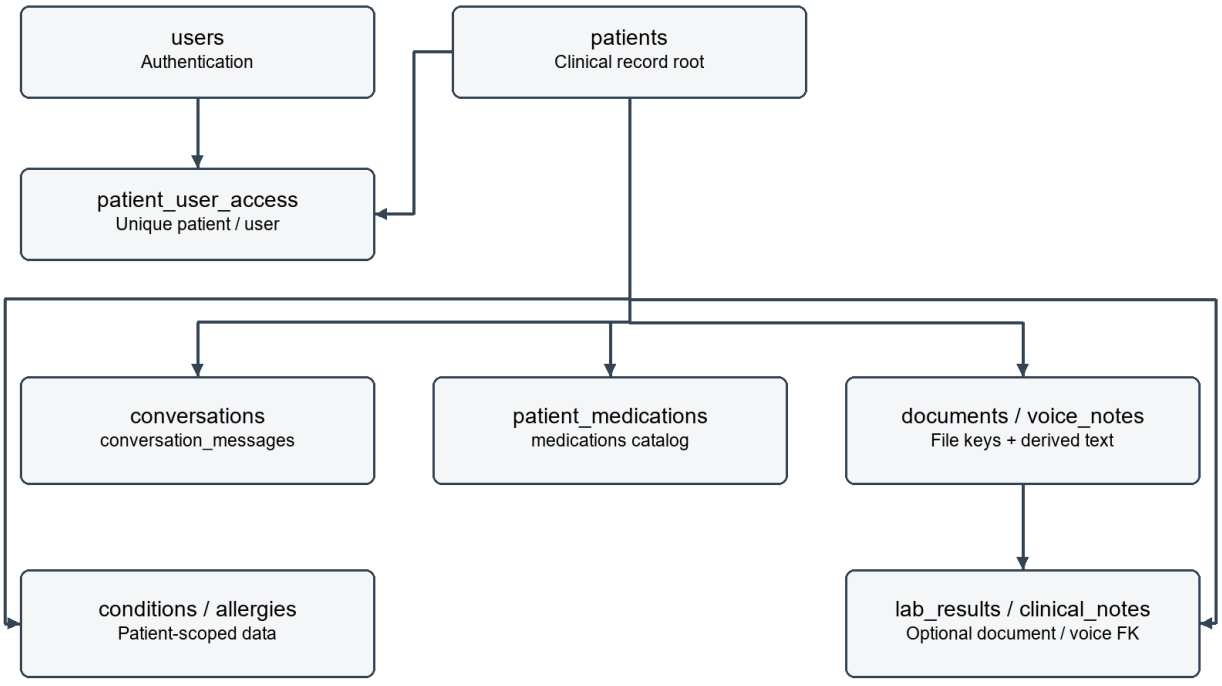
Condition یک مشکل بالینی متنی با status، onset_date، resolved_date و notes است. کاتالوگ بیماری یا کد تشخیصی استاندارد برای آن تعریف نشده است. Allergy نیز allergen، reaction و severity دارد و شدت‌های severe، moderate، mild و unknown را نگهداری می‌کند. فهرست مشکلات، فعال‌ها را

مقدم می‌گذارد و فهرست حساسیت، شدیدترها را. این ترتیب، تصمیم نمایش است و از آن نمی‌توان رتبه‌بندی قطعی خطر بیمار را نتیجه گرفت.

LabResult یک مدل عمومی برای مقدار منتخب است: test_name، value متنی، unit، reference_range، flag و result_date. نگهداری value به‌صورت متن نتایج کیفی را ممکن می‌کند، ولی محاسبه‌ی عددی یا مقایسه‌ی خودکار با بازه را بدون تبدیل اضافه فراهم نمی‌کند. وجود مدل به معنی وجود استخراج خودکار Hb یا WBC نیست. router فعلی مسیر CRUD مستقل آزمایش و یادداشت عمومی را ثبت نکرده است؛ یادداشت از صوت مسیر مشخص دارد و آزمایش در داده‌ی نمونه و Context دیده می‌شود.

۴-۵. روابط و رفتار حذف

شکل ۴-۱ رابطه‌های اصلی را نمایش می‌دهد. بیمار ریشه‌ی بیشتر داده‌ی بالینی است. ارتباط نسخه با Medication و ارتباط پیام با Conversation در زنجیره‌های جدا قرار دارند. PatientUserAccess پیوند user و patient را با قید یکتایی زوج برقرار می‌کند. این نمودار از FKهای مدل‌ها ساخته شده است و سرویس فرضی یا جدول برداری ندارد.



شکل ۴-۱ رابطه‌های اصلی پایگاه داده

ClinicalNote می‌تواند به VoiceNote با SET NULL مرتبط باشد. حذف ضبط، یادداشت تأییدشده را از بین نمی‌برد، ولی لینک آن را خالی می‌کند. LabResult نیز برای document_id رفتار SET NULL دارد. در مقابل پیام‌های یک گفتگو با حذف Conversation پاک می‌شوند. برای کاتالوگ دارو حذف cascade از نسخه وجود ندارد؛ مدخل ارجاع‌شده را نمی‌توان مانند فرزند بیمار حذف کرد. این تصمیم‌ها باید با سیاست نگهداری داده هماهنگ باشند.

ClinicalEvent به منبع با source_type و source_id اشاره می‌کند، اما این اشاره FK واقعی به چند جدول مختلف نیست. باز بودن نوع رویداد انعطاف دارد؛ هزینه‌ی آن نبود کنترل ارجاعی پایگاه برای source_id است. تایم‌لاین اصلی برای جلوگیری از اتکا به این جدول ناقص، از رکوردهای واقعی ساخته می‌شود. جدول رویداد همچنان برای برخی عملیات پر می‌شود و نباید آن را دفتر ممیزی کامل یا منبع انحصاری تایم‌لاین دانست.

۴-۶. نشست پایگاه داده و مهاجرت‌ها

Engine با pool_pre_ping و echo=False ساخته می‌شود. pre_ping پیش از استفاده، اتصال موجود را می‌آزماید و احتمال استفاده از اتصال قطع‌شده را کاهش می‌دهد؛ این ویژگی تضمین بازیابی هر تراکنش نیمه‌تمام نیست. SessionLocal با autoflush=False، autocommit=False و expire_on_commit=False تنظیم شده است. get_db با finally Session را می‌بندد. مدیریت commit و rollback در اختیار مسیر یا سرویس قرار دارد [۲].

هفت revision در Alembic زنجیره‌ی تغییر schema را ثبت می‌کنند: هسته‌ی بالینی، تحلیل سند، پاک‌سازی صوت، گفتگو، شواهد پیام، کاربر و دسترسی، و .OTP فایل‌های مهاجرت در فصل استقرار به‌عنوان گام جدا اجرا می‌شوند؛ create_all در fixture تست جای آزمودن واقعی migration را نمی‌گیرد. برای انتشار باید نسخه‌ی schema موجود و head مهاجرت یکسان باشند، و سازگاری downgrade فقط روی داده‌ی آزمایشی بررسی شود.

جدول ۳ زنجیره‌ی واقعی مهاجرت‌ها

Revision	پیشین	تغییر
d3d54628e9bd	آغاز	create_core_clinical_entities
b309d1be6900	d3d54628e9bd	add_document_ai_analysis_columns
fe6bf890d790	b309d1be6900	add_voice_note_cleanup_columns
03378aed5376	fe6bf890d790	add_persistent_conversations
ab22d66552c2	03378aed5376	add_message_evidence
9a1c78a56f1a	ab22d66552c2	add_users_and_patient_access
49c86fd969f2	9a1c78a56f1a	add_otp_login

۴-۷. خطا و یکپارچگی عملیات

خطر مهم در عملیات چندمرحله‌ای، موفقیت ناقص است. upload سند پیش از ساخت رکورد، شیء را ذخیره می‌کند؛ حذف سند نیز فایل را پیش از commit پایگاه حذف می‌کند. هر دو مسیر نیازمند رسیدگی جبرانی‌اند. در پردازش پس‌زمینه، SQLAlchemyError به rollback و log تبدیل می‌شود تا تغییر هم‌زمان

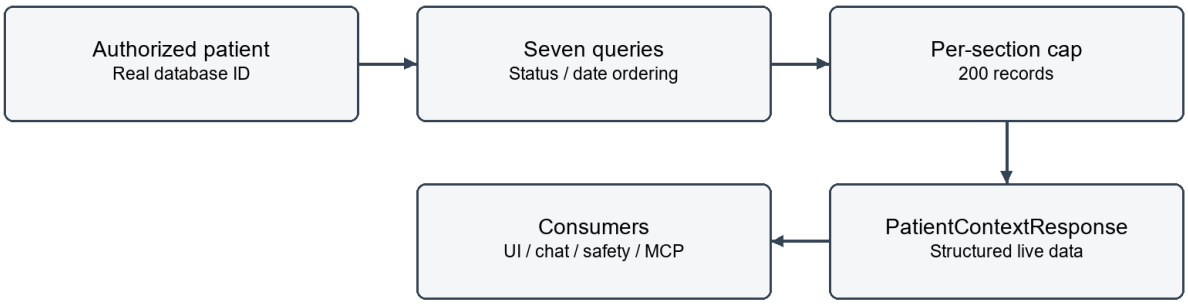
یا حذف رکورد باعث crash بی‌مهار نشود. با این حال صرف گرفتن exception به معنی اصلاح تمام وضعیت‌های نیمه‌کاره نیست.

برای توسعه‌ی بعدی می‌توان وضعیت کار، شناسه‌ی درخواست و نتیجه‌ی هر مرحله را در یک جدول عملیاتی نگهداری کرد. پردازش تکراری باید idempotent باشد، یعنی تکرار آن به داده‌ی ناسازگار یا نسخه‌ی تکراری منجر نشود. در نسخه‌ی فعلی idempotency key عمومی برای ارسال پیام، آپلود یا ایجاد یادداشت تعریف نشده است. بنابراین retry مرورگر پس از قطع پاسخ باید با احتیاط طراحی و آزمایش شود.

فصل پنجم پرونده بیمار زمینه و تایم‌لاین

۵-۱. تجمیع داده بدون تولید اطلاعات تازه

build_patient_context در services/patient_context.py هفت گروه داده‌ی بالینی را همراه مشخصات بیمار می‌خواند: مشکلات، داروها، حساسیت‌ها، آزمایش‌ها، اسناد، صوت و یادداشت‌ها. خروجی PatientContextResponse شامل generated_at است. این زمان، لحظه‌ی ساخت پاسخ را نشان می‌دهد؛ تاریخ ویزیت یا زمان رخداد همه‌ی اطلاعات نیست. در این سرویس تماس AI یا جست‌وجوی برداری وجود ندارد. شکل ۵-۱ مسیر تجمیع را نشان می‌دهد.



شکل ۵-۱ ساخت زمینه از رکوردهای مرتبط بیمار

هر query با patient_id محدود می‌شود. چنین فیلتر مشترکی لازمه‌ی جلوگیری از آمیخته شدن داده‌ی دو پرونده است، اما جای بررسی مجوز کاربر را نمی‌گیرد. بررسی مجوز پیش از ورود در route انجام می‌شود. مصرف‌کننده‌ی داخلی مانند MCP نیز از همین builder استفاده می‌کند، ولی اگر مستقیم اجرا شود احراز هویت وب را همراه ندارد. بنابراین patient-scoped بودن query و user-authorized بودن فراخوانی باید دو شرط جدا تلقی شوند.

ساخت زنده‌ی Context از انحراف یک نسخه‌ی کپی‌شده نسبت به پایگاه اصلی جلوگیری می‌کند. هزینه‌ی آن چند query در هر بار پرسش است. در نسخه‌ی فعلی لایه‌ی cache برای Context و invalidation متناظر دیده نمی‌شود. در مقیاس نمونه‌ی دانشگاهی این سادگی قابل فهم است؛ در داده‌ی بزرگ باید زمان queryها، حجم پاسخ و تعداد تماس‌های هم‌زمان اندازه‌گیری شود. انتخاب cache بدون سیاست تغییر داده می‌تواند همان مسئله‌ی کهنگی را بازگرداند.

۵-۲. ترتیب و سقف اطلاعات

در هر گروه سقف RECENT_LIMIT_ برابر دویست رکورد است. این سقف دفاعی اندازه‌ی فهرست را محدود می‌کند، اما انتخاب معنایی مرتبط‌ترین اطلاعات با سؤال نیست. برای مشکلات، active مقدم است و

سپس تاریخ شروع نزولی و id نزولی می‌آید. برای دارو، active، stopped و سایر وضعیت‌ها به ترتیب قرار می‌گیرند. حساسیت severe بر moderate و mild مقدم است. این ترتیب موجب می‌شود داده‌ی مهم‌تر از نظر وضعیت پیش از پر شدن سقف وارد شود.

آزمایش‌ها با result_date نزولی و تاریخ null در انتها خوانده می‌شوند. سند، صوت و یادداشت با created_at و id نزولی مرتب می‌شوند. selectinload برای مدخل دارو از query جدا برای هر نسخه جلوگیری می‌کند. استفاده از ترتیب دوم روی id نیز زمانی که تاریخ‌ها برابرند خروجی را قابل پیش‌بینی می‌سازد. این جزئیات برای تست و تجربه‌ی کاربر اهمیت دارند، زیرا تغییر بی‌دلیل ترتیب می‌تواند به‌صورت تغییر پرونده برداشت شود.

عنوان «زمینه‌ی کامل» در توضیح API باید با سقف دویست خوانده شود. اگر بیمار بیش از این مقدار رکورد در یک گروه داشته باشد، همه‌ی تاریخچه وارد پاسخ نمی‌شود. کد فیلد مشخصی برای اعلام تعداد حذف‌شده از Context ندارد. مدل نمی‌تواند از نبود یک رکورد در ورودی نتیجه بگیرد آن اتفاق هرگز رخ نداده است. بهتر است در توسعه‌ی بعدی علاوه بر محدوده‌ی زمانی، اطلاعات truncation نیز در قرارداد زمینه ثبت شود.

۵-۳. تفاوت نمای کلی بیمار با زمینه‌ی بالینی

get_patient_overview پاسخ کوچک‌تری برای نمای مرور بیمار دارد. مشکلات با active مقدم و حساسیت‌ها با severe مقدم‌اند؛ داروها فقط active هستند. نتایج آزمایش و یادداشت بالینی به پنج مورد اخیر محدود می‌شوند. این پاسخ با Context که سابقه‌ی دارو را نیز حمل می‌کند یکسان نیست. استفاده از overview برای پرسشی درباره‌ی داروی متوقف‌شده می‌تواند اطلاعات لازم را حذف کند؛ به همین دلیل مسیر AI از builder مستقل Context استفاده می‌کند.

واژه‌ی Summary در رابط به نمای ساختاریافته‌ی بیمار اشاره می‌کند. محتوای آن از رکوردها انتخاب می‌شود و برای ساختش مدل فراخوانی نمی‌شود. در مدل داده، ai_summary تحلیل روایی یک سند و cleaned_note متن مشتق از یک ضبط است. این خروجی‌ها به منبع مشخص خود وابسته‌اند و با خلاصه‌ی مستقل کل پرونده تفاوت دارند.

۵-۴. تایم‌لاین مشتق از رکورد

build_timeline برای مشکل، دارو، سند، صوت، یادداشت و آزمایش رویداد می‌سازد. برای Condition یک رویداد شروع و در صورت وجود resolved_date یک رویداد رفع می‌آید. برای PatientMedication شروع وجود دارد و پایان زمانی اضافه می‌شود که end_date موجود و status غیرactive باشد. سند و صوت با تاریخ ایجاد و آزمایش با result_date معرفی می‌شوند. حساسیت در حلقه‌های این تابع رویداد مستقل ندارد.

helper داخلی add، event_type، date، title، description، source_type و source_id را به TimelineItem تبدیل می‌کند. تاریخ اصلی اگر موجود نباشد با created_at جایگزین می‌شود. date بدون

ساعت در نیمه‌شب UTC قرار می‌گیرد و datetime بدون timezone، UTC فرض می‌شود. این قاعده برای مرتب‌سازی سازگار است، اما نمی‌تواند زمان واقعی رویداد مجهول را کشف کند. در نمایش باید معنای «تاریخ ثبت» از «تاریخ رخداد» قابل تشخیص باشد.

مرتب‌سازی نهایی روی سه‌تایی date، source_type و source_id با reverse=True انجام می‌شود. تساوی زمان‌ها به ترتیب قطعی منابع می‌انجامد، نه به ترتیب بالینی اثبات‌شده. این روش در برابر حذف رکورد نیز روشن است: رویداد مشتق آن در پاسخ بعدی وجود نخواهد داشت. رویداد دستی ClinicalEvent که منبعش در گروه‌های تابع نیست، صرفاً به دلیل بودن در آن جدول وارد این تایم‌لاین نمی‌شود.

جدول ۴ منبع و تاریخ رویدادهای تایم‌لاین

منبع	رویداد	تاریخ اصلی و fallback
Condition	condition_onset / condition_resolved	onset_date یا resolved_date؛ نبود تاریخ شروع → created_at
PatientMedication	medication_started / medication_stopped	start_date؛ پایان فقط با end_date و وضعیت غیرactive
Document	document_uploaded	created_at
VoiceNote	voice_note_recorded	created_at
ClinicalNote	note_created	created_at
LabResult	lab_result	result_date؛ در نبود → created_at

۵-۵. کیفیت داده و تفسیر نبود اطلاعات

Context وضعیت فعال و غیرفعال دارو، شدت حساسیت و تاریخ‌ها را حمل می‌کند، ولی اعتبار پزشکی آن‌ها را خودکار بازبینی نمی‌کند. اگر یک وضعیت اشتباه ثبت شده باشد، مدل همان داده را می‌بیند. اگر value آزمایش متن غیرقابل تبدیل باشد، builder آن را اصلاح نمی‌کند. این رفتار برای حفظ منبع مناسب است، اما ضرورت بازبینی داده‌ی ورودی را از بین نمی‌برد. تولید پاسخ دقیق‌تر به داده‌ی معتبر و کافی نیاز دارد.

نبود آلرژی ثبت‌شده با تأیید «فاقد آلرژی» متفاوت است. فهرست خالی فقط می‌گوید رکوردی در محدوده‌ی خوانده‌شده وجود ندارد. چنین تمایزی در داده‌ی نمونه و آزمون Context خالی اهمیت دارد. مدل باید نبود اطلاعات را بیان کند؛ prompt این رفتار را درخواست می‌کند، ولی schema به‌تنهایی معنای منفی بالینی را تعیین نمی‌کند. افزودن نشانگرهای صریح «پرسیده نشده» و «بررسی و رد شده» می‌تواند ابهام را کاهش دهد.

Context متن خام سند و رونوشت صوت را نیز حمل می‌کند. کاربر برای تبدیل رونوشت به ClinicalNote متن را بازبینی و ارسال می‌کند، اما مدل می‌تواند پیش از این مرحله transcript را در زمینه دریافت کند. پس تأیید یادداشت و دسترسی مدل به متن خام دو مرحله‌ی جدا هستند. افزودن وضعیت بازبینی به قرارداد زمینه می‌تواند به مصرف‌کننده نشان دهد کدام متن هنوز تأیید نشده است.

۵-۶. آزمون‌های زمینه و مسیر آینده

test_patient_context_api.py و test_patient_overview_api.py انتظار حضور گروه‌ها، ترتیب و داده‌ی اخیر را تعریف می‌کنند. test_patient_context_api.py همچنین تایم‌لاین و fallback تاریخ را بررسی می‌کند. این آزمون‌ها به PostgreSQL fixture متکی‌اند و در اجرای منتخب این گزارش اجرا نشده‌اند. شواهد آن‌ها وجود قرارداد آزموده‌شدنی است؛ نتیجه‌ی فعلی آن‌ها باید با اجرای مستقل روی پایگاه آزمایشی گزارش شود.

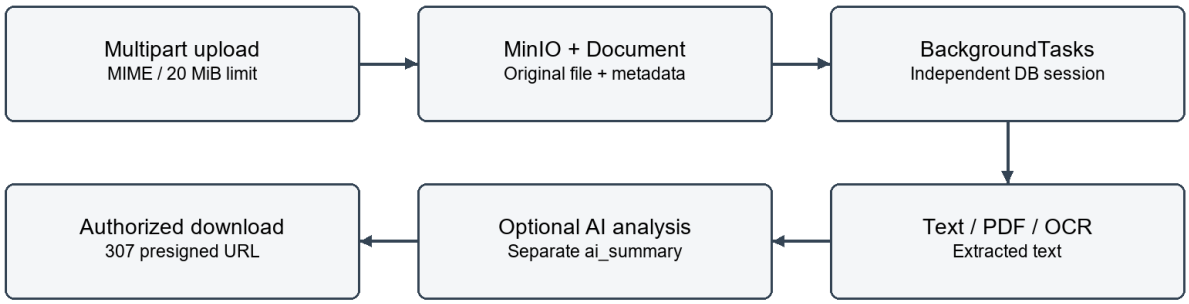
بهبود مناسب Context، انتخاب کنترل‌شده‌ی داده بر اساس سؤال و حفظ اطلاعات حذف‌شده است. می‌توان پرسش درباره‌ی دارو را به سابقه‌ی مصرف و حساسیت محدود کرد، اما این انتخاب باید قابل توضیح باشد و اطلاعات حساس لازم را کنار نگذارد. روش انتخاب باید در کنار حالت «پرونده‌ی کامل‌تر» ارزیابی شود. صرف کاهش تعداد token کافی نیست؛ باید نرخ حذف شواهد لازم و اثر آن بر پاسخ اندازه‌گیری شود.

فصل ششم بارگذاری پردازش و تحلیل اسناد

۶-۱. قرارداد بارگذاری و ثبت متادیتا

مسیر POST /api/v1/patients/{patient_id}/documents فایل را با فیلد file و title و doc_type اختیاری دریافت می‌کند. بیمار از وابستگی مجوزدار می‌آید. نام فایل حداکثر سیصد نویسه و حجم حداکثر بیست MiB است. نوع‌های مجاز PDF، PNG، JPEG، WebP و متن ساده‌اند. فایل خالی و نوع خارج از فهرست به ۴۲۲ تبدیل می‌شوند. MIME از مشخصات UploadFile خوانده می‌شود؛ بررسی امضای واقعی بایت فایل به‌عنوان کنترل جدا پیاده نشده است.

پس از read کامل، storage.upload_file بایت‌ها را به MinIO می‌فرستد. رکورد Document عنوان، نوع، کلید، نام اولیه، MIME و file_size_bytes را نگهداری می‌کند. flush شناسه را فراهم می‌سازد و یک ClinicalEvent برای بارگذاری ایجاد می‌شود. سپس commit و refresh انجام می‌گیرد و process_document با شناسه در BackgroundTasks قرار می‌گیرد. شکل ۶-۱ نقاط ذخیره و تولید داده‌ی مشتق را نشان می‌دهد.



شکل ۶-۱ جریان سند از فایل اولیه تا متن و تحلیل اختیاری

۶-۲. کلید شیء و مسئولیت ذخیره‌ساز

build_storage_key اجزای مسیر ورودی را حذف می‌کند، نویسه‌های نامطمئن را با زیرخط جایگزین می‌کند، نقطه‌های پی‌درپی را جمع می‌کند و نام را تا صد و پنجاه نویسه محدود می‌سازد. مسیر نهایی شامل patients، شناسه‌ی بیمار، پوشه و UUID است. استفاده از شناسه‌ی یکتا احتمال برخورد نام را کاهش می‌دهد و مانع می‌شود نام تکراری فایل قبلی را بازنویسی کند. وجود شناسه‌ی بیمار در مسیر، مجوز نیست و حفاظت باید در مسیر دسترسی اعمال شود.

get_client نمونه‌ی MinIO را به‌صورت lazy می‌سازد. در upload، bucket در صورت نبود ایجاد می‌شود. get_object_bytes پاسخ را در finally می‌بندد و اتصال را آزاد می‌کند. خطاهای S3Error به

StorageError تبدیل می‌شوند تا route با جزئیات SDK درگیر نباشد. با این حال همه‌ی خطاهای transport الزاماً S3Error نیستند؛ نتیجه‌ی اجرای تست ذخیره‌سازی نشان داد قطع اتصال می‌تواند در fixture به exception دیگری برسد.

presigned_download_url به‌طور پیش‌فرض یک ساعت اعتبار دارد. این نشانی یک مجوز موقت مبتنی بر امضا است و تا پایان اعتبار می‌تواند توسط دارنده‌ی آن استفاده شود. خروجی مدل نباید این URL را تولید کند و URL در evidence پایدار ذخیره نمی‌شود. این انتخاب از کهنه شدن لینک‌های ذخیره‌شده جلوگیری می‌کند، ولی سیاست اعتبار و امکان لغو پس از تغییر دسترسی همچنان نیازمند بررسی عملیاتی است.

۶-۳. استخراج متن و وضعیت کار

process_document Session مستقل می‌سازد و Document را با id می‌خواند. نبود رکورد با log و بازگشت مدیریت می‌شود. ابتدا extraction_status به processing می‌رود و commit می‌شود. فایل از ذخیره‌ساز خوانده و extract_text اجرا می‌شود. متن strip‌شده یا None و وضعیت completed ثبت می‌شوند. completed در اینجا پایان اجرای ابزار است و لزوماً به معنی متن غیرخالی یا استخراج صحیح تمام محتوای سند نیست.

در شکست عمومی، متن پاک و وضعیت failed ثبت می‌شود؛ در SQLAlchemyError rollback انجام می‌گیرد. خطای روش استخراج در log دیده می‌شود، ولی Document فیلد error_message مانند VoiceNote ندارد. روش ExtractionResult نیز در log ثبت می‌شود و ستون مستقل method در مدل Document دیده نمی‌شود. بنابراین اطلاعات منشأ موجود شامل فایل و ارتباط رکورد است، نه یک گزارش پایدار کامل از نسخه‌ی OCR و روش هر صفحه.

مسیر extract برای اجرای مجدد ۲۰۲ می‌دهد و اگر وضعیت processing باشد ۴۰۹ برمی‌گرداند. کنترل فقط روی وضعیت جاری است و enqueue هم‌زمان پیش از تغییر آن به processing هنوز می‌تواند به بررسی رقابت نیاز داشته باشد. رابط باید وضعیت را دوباره بخواند؛ این پاسخ به معنی پایان استخراج نیست. برای صف پایدار، لازم است شناسه‌ی job و انتقال وضعیت در یک قرارداد اتمیک تعریف شود.

۶-۴. استخراج متن PDF و تشخیص نوری نویسه‌ها (OCR)

برای text/plain، UTF-8 با errors=replace استفاده می‌شود. بایت نامعتبر باعث توقف decoding نمی‌شود، اما ممکن است نویسه‌ی جایگزین در متن ایجاد کند. PDF با pypdf خوانده و فقط پنجاه صفحه‌ی نخست وارد پردازش می‌شود. متن صفحات به هم متصل می‌گردد و اگر طول مجموع آن حداقل پنجاه نویسه باشد روش pdf_text انتخاب می‌شود. آستانه بر مجموع سند است؛ برای هر صفحه‌ی کم‌متن به‌طور مستقل OCR اجرا نمی‌شود.

در PDF ترکیبی، ممکن است صفحه‌ی اول متن کافی داشته باشد و صفحات اسکن‌شده‌ی بعدی بدون متن بمانند. این محدودیت از شرط مجموع متن نتیجه می‌شود. اگر مجموع کوتاه باشد، تصاویر جاسازی‌شده‌ی صفحات برای OCR استفاده می‌شوند. این مسیر با رندر تمام صفحه به تصویر فرق دارد؛ چیدمان چند تصویر، متن برداری یا سند غیرمعمول می‌تواند پوشش متفاوتی داشته باشد. مستندات pypdf نیز تمایز استخراج لایه‌ی متن و OCR را توضیح می‌دهد [۸].

Tesseract در اولین نیاز بررسی و نتیجه‌ی آماده بودن cache می‌شود. اگر هر دو fas و eng نصب باشند زبان fas+eng انتخاب می‌شود و در غیر این صورت یکی از زبان‌های موجود استفاده می‌شود. تصویر با Pillow خوانده می‌شود و pytesseract متن می‌دهد. نبود binary با ExtractionUnavailable بیان می‌شود. نصب بسته‌ی Python به‌تنهایی OCR را فعال نمی‌کند؛ executable و بسته‌ی زبان هم لازم‌اند.

کیفیت OCR به خوانایی فایل و ترتیب خروجی وابسته است. کد فعلی validator واحدهای آزمایشگاهی، confidence هر واژه یا تصحیح عدد با فایل اولیه ندارد. در تست تصویر فارسی تنها تولید حداقل متن معنادار از نظر طول بررسی می‌شود؛ این آزمون دقت نویسه یا تطابق محتوای پزشکی را نمی‌سنجد. حفظ فایل اولیه امکان بازبینی را می‌دهد، اما جای سنجش خطای استخراج را نمی‌گیرد.

۶-۵. تحلیل سند با هوش مصنوعی

تحلیل در /api/v1/documents/{document_id}/analyze POST با خواندن سند و بررسی دسترسی آغاز می‌شود. شرط اجرای آن completed بودن استخراج و غیرخالی بودن extracted_text است؛ در غیر این صورت ۴۰۹ داده می‌شود. analyze_document با ارائه‌دهنده‌ی مشترک AI یک پیام از عنوان، نوع و متن سند می‌سازد. درخواست، خلاصه‌ای فارسی با یافته‌های موجود و محدودیت‌ها می‌خواهد و از جعل یافته، عدد، تشخیص یا داروی تازه منع می‌کند.

خروجی متن در ai_summary ذخیره می‌شود و analysis_status به completed یا failed می‌رود. اصل فایل و extracted_text عوض نمی‌شوند. اگر AI تنظیم نشده باشد ۵۰۳ برمی‌گردد؛ اگر AIError رخ دهد mark_analysis_failed خلاصه را خالی و وضعیت را failed می‌کند و route خطای ۵۰۲ می‌دهد. این خطا درباره‌ی تحلیل است و نباید به‌عنوان خرابی فایل اصلی نمایش داده شود.

خروجی تحلیل سند، متن آزاد ai_summary است. سرویس از این متن رکورد LabResult نمی‌سازد؛ ثبت مقدار آزمایش به طرح‌واره، واحد، منبع و بازبینی جدا نیاز دارد. پروپوزال استخراج فیلدهای منتخب را هدف گرفته بود، در حالی که مسیر فعلی تا استخراج متن و تحلیل سطح سند پیش رفته است. برای نمونه، اشاره‌ی مدل به یک مقدار در خلاصه، آن مقدار را به داده‌ی ساختاریافته‌ی ثبت‌شده تبدیل نمی‌کند.

۶-۶. دانلود حذف و نقاط یکپارچه‌سازی

GET download ابتدا get_document_or_404_ را اجرا می‌کند؛ این helper مجوز بیمار را بررسی می‌کند. سپس URL امضاشده ساخته و پاسخ ۳۰۷ داده می‌شود. آزمون دسترسی نیز ترتیب مجوز پیش از تولید URL را بررسی می‌کند. مسیر PATCH عنوان و نوع را تغییر می‌دهد و DELETE شیء و رکورد و ClinicalEvent متناظر را حذف می‌کند. موفقیت حذف در دو سامانه تراکنش واحد ندارد.

رابط اسناد، بارگذاری و فهرست فایل‌ها و وضعیت استخراج و تحلیل را نمایش می‌دهد. آزمون سرتاسری این گردش کار باید انتقال مجوز در دانلود، هدایت ۳۰۷ و دسترسی مرورگر به hostname ذخیره‌ساز را نیز پوشش دهد. شکاف پیوند مستقیم EvidenceList در بخش ۴-۳ توضیح داده شد؛ تست API تنها بخش سمت سرور همین جریان را می‌سنجد.

۶-۷. آزمون‌های پردازش سند و مسیر توسعه

در اجرای منتخب هفت تست extraction موفق شدند: متن ساده، PDF با لایه‌ی متن، نبود OCR، OCR تصویر انگلیسی و فارسی و نوع نامجاز در میان آن‌هاست. این نتیجه نشان می‌دهد مسیرهای بررسی‌شده در محیط موجود کار کرده‌اند. آزمون‌های API اسناد و تحلیل سند در این اجرای محدود انتخاب نشده‌اند و نتیجه‌ی موفق فعلی برای آن‌ها ادعا نمی‌شود.

برای مرحله‌ی بعد می‌توان مجموعه‌ای از اسناد مصنوعی با متن مرجع ساخت: سند کاملاً متنی، کاملاً اسکن‌شده، ترکیبی، جدول چندستونی، متن فارسی و لاتین و سند بیش از سقف صفحه. باید هم پوشش صفحات و هم خطای محتوا اندازه‌گیری شود. استخراج فیلد نیازمند schema، منبع هر مقدار، وضعیت بازبینی و آزمون روی واحد و عدد است. افزودن نام چند فیلد به prompt، جای این زنجیره‌ی اعتبارسنجی را نمی‌گیرد.

فصل هفتم مدل زبانی و گفتگوی بالینی

۷-۱. قرارداد ارائه‌دهنده و پیکربندی

LLMProvider یک Protocol با complete است. ورودی‌های آن system، user و history اختیاری و خروجی آن str است. سه پیاده‌سازی موجود عبارت‌اند از OpenAICompatibleProvider، GeminiProvider و .OpenRouterProvider درخواست با httpx ساخته می‌شود و SDK مدل شرط اجرا نیست. قرارداد کوچک باعث می‌شود تحلیل سند، پاک‌سازی صوت، گفتگو و ایمنی ساختاریافته از همان لایه استفاده کنند، هرچند prompt و قرارداد خروجی‌شان متفاوت است.

build_provider شناسه‌ی provider و model را از درخواست یا تنظیمات می‌گیرد. شناسه‌ی ارائه‌دهنده باید در PROVIDER_IDS باشد. model طول حداکثر دویست دارد و URL یا رشته‌ی دارای فاصله پذیرفته نمی‌شود. کلید و base URL از کاربر دریافت نمی‌شوند. در OpenRouter کلید اختصاصی OPENROUTER_API_KEY مقدم و AI_API_KEY جایگزین است؛ سایر مسیرهای متن از AI_API_KEY استفاده می‌کنند. نبود کلید AIUnavailableError ایجاد می‌کند.

get_provider نمونه‌ی پیش‌فرض محیط را cache می‌کند و انتخاب صریح درخواست از build_provider می‌گذرد. get_settings نیز cache دارد؛ تغییر فایل محیط در فرایند فعال لزوماً تنظیمات را تازه نمی‌کند. reset_provider_cache برای آزمون وجود دارد. در استفاده‌ی عادی باید اثر restart و متغیر build-time رابط از تنظیمات runtime سرور جدا توضیح داده شود.

جدول ۵ قرارداد ارائه‌دهنده‌های متن

ارائه‌دهنده	قالب تماس	کلید و محدودیت
OpenAICompatibleProvider	chat/completions و messages	AI_API_KEY؛ base URL از سرور
GeminiProvider	generateContent و system_instruction	AI_API_KEY؛ roles به مدل نگاشت می‌شوند
OpenRouterProvider	chat/completions و X-Title	OPENROUTER_API_KEY سپس AI_API_KEY؛ نام مدل تضمین availability نیست

۷-۲. درخواست به ارائه‌دهنده‌های OpenAI-compatible و OpenRouter

OpenAICompatibleProvider فهرست messages را با system، history و پیام user می‌سازد و به /chat/completions می‌فرستد. model در JSON و کلید در Authorization قرار می‌گیرد. پاسخ از choices[0].message.content خوانده می‌شود. statusهای ۴۰۱ و ۴۰۳ به رد کلید، ۴۲۹ به محدودیت نرخ و سایر وضعیت‌های غیر۲۰۰ به AIError تبدیل می‌شوند. نبود ساختار مورد انتظار نیز به خطای تمیز تبدیل می‌گردد.

OpenRouterProvider قالب درخواست را از OpenAICompatibleProvider به ارث می‌برد و base URL و X-Title را تعیین می‌کند. شناسه‌ی پیش‌فرض مخزن deepseek/deepseek-r1:free است. این مقدار پیکربندی است و دسترس‌پذیری زنده‌ی آن در اجرای حاضر آزموده نشده است. مستندات رسمی OpenRouter قرارداد عمومی API را شرح می‌دهد [۶]؛ انتخاب مدل قابل استفاده باید با حساب و قابلیت مورد نیاز سازگار باشد.

مدل انتخابی در تمام گردش کارها از رابط اعمال نمی‌شود. chat و conversation فیلد provider و model می‌پذیرند، اما تحلیل سند، پاک‌سازی و endpoint اختصاصی ایمنی از get_provider پیش‌فرض استفاده می‌کنند. حالت agent هم از تنظیم پیش‌فرض می‌گیرد. بنابراین انتخاب یک مدل در selector گفتگو نباید در گزارش به‌عنوان تغییر مدل تمام سامانه معرفی شود.

۷-۳. قالب پیام و پاسخ در Gemini

GeminiProvider از models/{model}:generateContent در v1beta استفاده می‌کند. system در system_instruction قرار می‌گیرد و history با roleهای Gemini تبدیل می‌شود؛ assistant به model نگاشت می‌گردد. محتوا به‌صورت parts دارای text ارسال و پاسخ از candidates[0].content.parts به هم متصل می‌شود. کلید در x-goog-api-key قرار دارد. مستندات Google روش تولید متن را توضیح می‌دهد [۵]؛ قالب خاص این client از کد برداشت شده است.

برای ۴۰۳ HTML یک تشخیص جدا وجود دارد تا مسدود شدن در سطح شبکه با خطای کلید یکسان تلقی نشود. در ۴۰۰ با متن «API key not valid» رد کلید تشخیص داده می‌شود. این شاخه‌ها برای بیان علت در log مفیدند، اما معیار جامع تشخیص همه‌ی محدودیت‌های شبکه نیستند. API به کاربر پیام فارسی عمومی می‌دهد و علت اصلی در سمت سرور ثبت می‌شود.

مقدار gemini-3.6-flash در پیش‌فرض و کاتالوگ ثابت مخزن آمده است. کاتالوگ Gemini از سرویس کشف مدل دریافت نمی‌شود؛ در نتیجه نام‌های آن ممکن است با مدل‌های قابل استفاده‌ی حساب همگام نباشند. صحت نام و دسترس‌پذیری باید پیش از اجرای واقعی بررسی شود و خطای مدل ناموجود از مسیر مدیریت پاسخ ارائه‌دهنده به کاربر برسد.

۷-۴. تکرار تماس و خطای شبکه

_post_with_retries برای خطاهای transport حداکثر سه تلاش دارد و میان آن‌ها یک و دو ثانیه منتظر می‌ماند. پاسخ HTTP خطا به‌خودی‌خود در این helper retry نمی‌شود؛ مثلاً ۴۲۹ از مسیر تفسیر پاسخ به AIError می‌رود. timeout از AI_TIMEOUT_SECONDS می‌آید. چند تلاش می‌تواند زمان نهایی را بیش از timeout یک تماس کند. بودجه‌ی کل و deadline سراسری برای گردش کار تعریف نشده است.

هر provider برای complete یک httpx.Client می‌سازد و در finally آن را می‌بندد. AI_PROXY به این client منتقل می‌شود. log درخواست حضور کلید را به‌صورت boolean بیان می‌کند، ولی _log_response تا هزار نویسه‌ی متن پاسخ را در سطح info ثبت می‌کند. این متن ممکن است اطلاعات بالینی تولیدشده داشته باشد. پس عدم چاپ کلید به معنی عدم ثبت اطلاعات بیمار در log نیست؛ پاک‌سازی و سیاست نگهداری log لازم است.

۷-۵. ساخت ورودی مدل و محدودیت زمینه

build_user_message یک JSON با ensure_ascii=False و سپس سؤال پزشک می‌سازد. متن extracted_text، transcript و content یادداشت هرکدام تا دو هزار نویسه کوتاه می‌شوند. سقف به هر فیلد مربوط است و محدودیت token کل پیام نیست. هفت گروه دویست‌تایی همراه metadata و متن‌های مشتق دیگر می‌توانند ورودی بزرگی بسازند. ai_summary و cleaned_note در این helper به همان روش محدود نمی‌شوند.

در قطع متن، علامت حذف اضافه می‌شود ولی انتخاب قسمت مرتبط با سؤال انجام نمی‌شود. اگر بخش لازم در پایان یک سند باشد ممکن است وارد مدل نشود. از سوی دیگر JSON پرونده مشخصات پایه را نیز حمل می‌کند و ناشناس‌سازی پیش از ارسال در این مسیر دیده نمی‌شود. کنترل زمینه‌ی آینده باید اندازه، ارتباط و محرمانگی را هم‌زمان پوشش دهد و حد صرفاً نویسه‌ای را معادل محافظت کامل تلقی نکند.

۷-۶. گفتگوی بدون ماندگاری

POST /api/v1/patients/{patient_id}/chat سؤال، تاریخچه و فلگ medication_safety را می‌گیرد. سؤال طول یک تا دو هزار و هر پیام تاریخچه تا چهار هزار نویسه دارد؛ فهرست تاریخچه حداکثر بیست ورودی می‌پذیرد. route از انتهای تاریخچه به اندازه‌ی AI_MAX_HISTORY_TURNS استفاده می‌کند که پیش‌فرض شش است. با وجود نام turns، برش روی تعداد پیام انجام می‌شود و الزاماً شش زوج سؤال و پاسخ نیست.

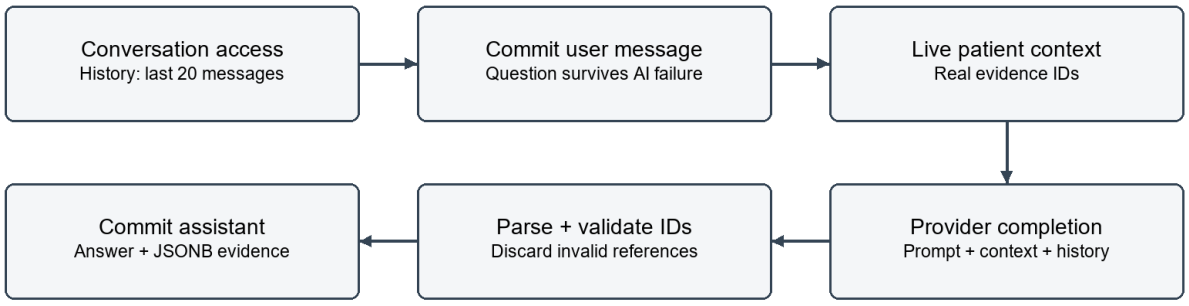
Context پس از مجوز ساخته می‌شود و prompt معمول یا ایمنی انتخاب می‌گردد. نتیجه ChatResponse شامل patient_id، question، answer و generated_at است. این route پیام را در

پایگاه نگهداری نمی‌کند و evidence ساختاریافته‌ی مسیر ماندگار را نیز اجرا نمی‌کند. هدف آن قرارداد ساده‌ی اولیه و سازگاری است؛ شرح Citation باید به مسیر درست نسبت داده شود.

۷-۷. گفتگوی ماندگار و ترتیب ثبت پیام‌ها

Conversation به بیمار و ConversationMessage به گفتگو وابسته‌اند. عنوان پیش‌فرض «گفتگوی جدید» است و نخستین پیام در نبود عنوان صریح به عنوان کوتاه تا پنجاه نویسه تبدیل می‌شود. این عنوان‌سازی قطعی است و مدل دیگری فراخوانی نمی‌کند. فهرست گفتگو بر updated_at و id نزولی و پیام‌ها بر id صعودی ارائه می‌شوند. شواهد پیام دستیار در JSONB ذخیره می‌گردند.

send_conversation_message ابتدا مجوز گفتگو را می‌سنجد و بیست پیام اخیر پیش از پیام جاری را خوانده، ترتیب زمانی آن‌ها را بازمی‌گرداند. سپس پیام user و در صورت لزوم عنوان را commit می‌کند. بعد provider انتخاب می‌شود و Context تازه، index شواهد و annotation ساخته می‌شوند. تماس مدل انجام می‌شود، پاسخ و شناسه‌ها parse و validate می‌شوند و پیام assistant همراه شواهد commit می‌شود. شکل ۷-۱ این دو مرز ثبت را مشخص می‌کند.



شکل ۷-۱ توالی گفتگوی ماندگار و حفظ سؤال در شکست مدل

اگر انتخاب provider نامعتبر یا سرویس تنظیم‌نشده باشد نیز پیام کاربر پیش‌تر ثبت شده است. در شکست AI پاسخ ۵۰۲ می‌آید و پیام دستیار ساخته نمی‌شود. retry کاربر می‌تواند پیام تازه‌ی تکراری بسازد؛ شناسه‌ی درخواست برای جلوگیری از آن وجود ندارد. به‌علاوه فراخوانی‌های هم‌زمان ممکن است پاسخ‌ها را با ترتیب پایان ذخیره کنند. ترتیب id در نتیجه، ترتیب کامل و قطعی زوج‌های مکالمه تحت رقابت نیست و نیاز به آزمون دارد.

۷-۸. کاتالوگ و رابط گفتگو

GET /api/v1/ai/providers احراز هویت می‌خواهد و فقط شناسه و نام ارائه‌دهنده و مدل را عرضه می‌کند. Gemini از فهرست ثابت و OpenRouter از models/ خوانده می‌شود. نتیجه‌ی کشف تا سیصد ثانیه cache می‌گردد و شکست کشف مدل خیالی تولید نمی‌کند. با این حال حضور کاتالوگ عمومی OpenRouter به‌تنهایی به معنی آماده بودن کلید برای completion نیست و build_provider دوباره کلید را می‌سنجد.

رابط گفتگوی بالینی انتخاب‌گر مدل و دو حالت ایمنی و عامل آزمایشی دارد. انتخاب‌ها در localStorage نگهداری می‌شوند و برای پیام ماندگار، provider و model به API ارسال می‌شوند. متن پاسخ و شواهد در مؤلفه‌های جدا نمایش داده می‌شوند. مالکیت اصلی این رابط مطابق پروپوزال با ایمان است و اتصال آن به قراردادهای سمت سرور در حوزه‌ی یکپارچه‌سازی مشترک قرار دارد.

۷-۹. دامنه‌ی تضمین و ارزیابی لازم

قواعد prompt از جعل داده و تصمیم مستقل منع می‌کنند، اما اعتبارسنجی فعلی روی متن آزاد نمی‌تواند هر ادعای پزشکی را اثبات یا رد کند. در مسیر evidence اعتبار شناسه کنترل می‌شود و در ایمنی JSON نوع فیلدها؛ این دو کنترل مکمل‌اند ولی معیار صحت پزشکی نیستند. برای ارزیابی باید پاسخ‌ها بر اساس شواهد پرونده و دانش مرجع بررسی شوند و نرخ ادعای بدون پشتوانه به‌صورت مستقل گزارش گردد.

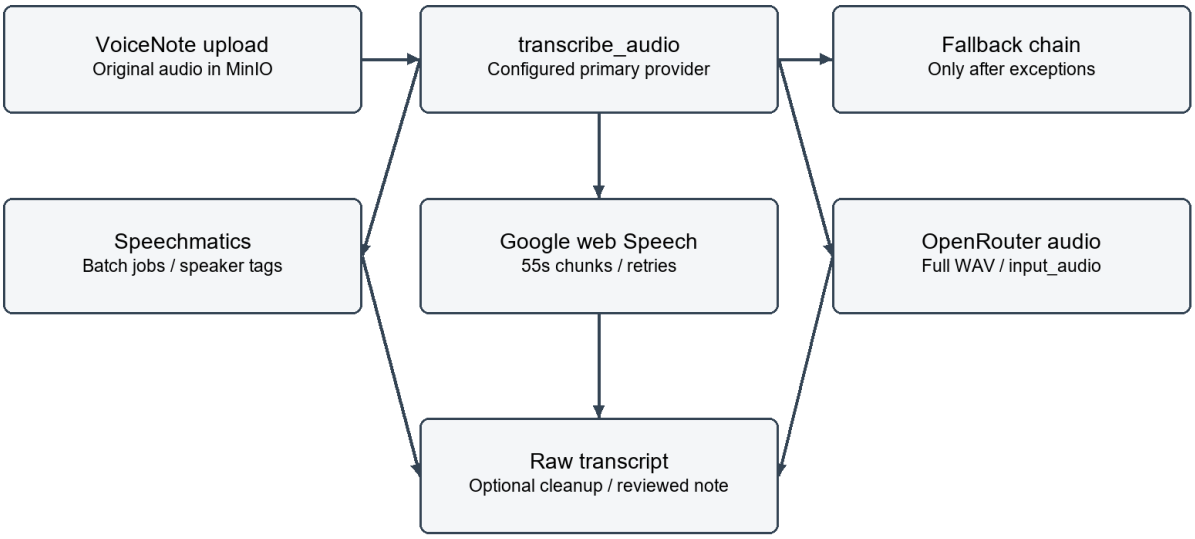
test_openrouter_provider.py در اجرای منتخب یازده آزمون موفق داشت و ساخت درخواست، خطا و parse پاسخ را با MockTransport بررسی کرد. این نتیجه موفقیت قرارداد کد است، نه موفقیت سرویس زنده یا صحت پاسخ بالینی. تست انتخاب catalog و APIهای گفتگو در مخزن وجود دارد ولی در این اجرا انتخاب نشده است. گزارش‌های تسک قبلی نیز شکست واقعی شبکه‌ی Gemini را ذکر کرده‌اند و نباید به پاسخ موفق تبدیل شوند.

فصل هشتم صوت و تبدیل گفتار به متن

۸-۱. ثبت صوت و وضعیت‌های مستقل

آپلود صوت در مسیر وابسته به بیمار با عنوان، conversation_type و duration_seconds اختیاری انجام می‌شود. MIME از پارامترهای بعد از نقطه‌ویرگول جدا می‌شود تا خروجی MediaRecorder مانند audio/webm دارای codecs پذیرفته شود. فهرست مجاز شامل WebM، OGG، MP3، MP4/M4A، WAV و FLAC و نیز video/webm است. سقف Backend پنجاه MiB و نام فایل سیصد نویسه است. duration_seconds از فرم دریافت می‌شود و اندازه‌گیری معتبر مدت در این route انجام نمی‌شود.

اصل صوت در پوشه‌ی voice از ذخیره‌ساز قرار می‌گیرد. VoiceNote شامل زبان، نوع مکالمه، وضعیت رونویسی، transcript، وضعیت پاک‌سازی، cleaned_note، error_message و processed_at است. پردازش پس‌زمینه ابتدا processing و سپس completed یا failed را ثبت می‌کند. در اجرای دوباره، cleaned_note پاک و cleanup_status pending می‌شود؛ خروجی مشتق قبلی دیگر متناظر با ورودی تازه فرض نمی‌شود.



شکل ۸-۱ ارائه‌دهنده‌های گفتار و مسیرهای جایگزین

۸-۲. قرارداد STT و انتخاب ارائه‌دهنده

SpeechToTextProvider دارای transcribe با data، file_name، mime_type، language و diarize است و متن برمی‌گرداند. facade عمومی transcribe_audio نام provider را از STT_PROVIDER می‌گیرد. پیش‌فرض کد speechmatics است. مسیرهای google و openrouter در

فایل کاری فعلی وجود دارند و زنجیره‌ی STT_FALLBACK_PROVIDERS می‌تواند پس از مسیر اصلی اجرا شود. این طراحی انتخاب سرویس را از route صوت جدا می‌کند.

نتیجه‌ی قرارداد فقط متن است. شناسه‌ی provider موفق، زمان‌بندی واژه‌ها، confidence و جزئیات job در مدل VoiceNote ذخیره نمی‌شوند. بنابراین وقتی fallback موفق شود، متن نهایی در پایگاه لزوماً نشان نمی‌دهد کدام مسیر آن را ساخته است. ثبت metadata ارائه‌دهنده در آینده برای ارزیابی و ردیابی اهمیت دارد. همچنین تمام providerها تضمین یکسانی درباره‌ی گوینده یا متن خالی ندارند.

۸-۳. پردازش دسته‌ای صوت در Speechmatics

SpeechmaticsProvider ابتدا job_config با type=transcription، language و operating_point=enhanced می‌سازد. اگر diarize درخواست شود، diarization=max افزوده می‌شود. فایل و JSON تنظیمات به jobs/ POST می‌روند. پاسخ ۲۰۱ باید id کار را داشته باشد. سپس GET jobs/{id}/ در فاصله‌ی پیش‌فرض سه ثانیه تا deadline پیش‌فرض سیصد ثانیه تکرار می‌شود. done پایان موفق و error، rejected یا cant_run شکست است.

متن از jobs/{id}/transcript/ با format=json-v2 گرفته می‌شود. format_transcript_json نخستین alternative هر بخش را می‌خواند و هنگام تغییر speaker خط جدید با برچسب می‌سازد. برچسب S1 یا S2 به‌تنهایی معلوم نمی‌کند فرد پزشک است یا بیمار. نسبت دادن نقش نیازمند اطلاعات یا بازبینی بیشتر است. ورودی doctor_patient سبب درخواست diarize می‌شود؛ physician_note آن را فعال نمی‌کند.

این مسیر یک job غیرهمگام در سرویس خارجی دارد، ولی worker Backend تا نتیجه منتظر می‌ماند. صف پایدار مستقل برای پیگیری job پس از restart موجود نیست و شناسه‌ی job هم در VoiceNote ثبت نمی‌شود. پاسخ خالی می‌تواند به transcript=None با وضعیت completed تبدیل شود. completed یعنی کار اجرا شده است؛ نیاز به بازبینی کفایت متن همچنان وجود دارد.

۸-۴. آماده‌سازی فایل و تشخیص گفتار با Google web Speech

GoogleSpeechProvider از SpeechRecognition و recognize_google استفاده می‌کند؛ این مسیر Google Cloud Speech-to-Text با حساب سرویس نیست. زبان fa به fa-IR نگاشت می‌شود. ffprobe مدت فایل را می‌خواند و ffmpeg آن را به WAV تک‌کاناله با نرخ شانزده هزار نمونه در ثانیه تبدیل می‌کند. این تبدیل شکل ورودی را یکنواخت می‌کند؛ ادعای افزایش دقت از آن بدون آزمایش قابل نتیجه‌گیری نیست.

اگر مدت نامعلوم یا کمتر از سقف باشد یک فایل WAV ساخته می‌شود. برای مدت بیشتر، بازه‌ها از صفر با گام STT_GOOGLE_MAX_CHUNK_SECONDS، پیش‌فرض پنجاه‌وپنج ثانیه، ایجاد می‌شوند. در آزمون صوت ۱۲۰ ثانیه‌ای سه شروع صفر، ۵۵ و ۱۱۰ انتظار می‌رود. قطعه‌ها هم‌پوشانی ندارند و متن خروجی با

فاصله به هم متصل می‌شود. بریدن واژه روی مرز قطعه و از دست رفتن زمینه‌ی جمله باید در ارزیابی واقعی بررسی شود.

برای هر قطعه، UnknownValueError باعث عبور از آن قطعه می‌شود. سایر exceptionها در حلقه‌ی تلاش، تا STT_GOOGLE_RETRIES، با پیش‌فرض سه بار، دوباره امتحان می‌شوند و شکست آخر به فراخواننده می‌رسد. اگر فهرست texts خالی بماند، TranscriptionError فارسی ایجاد می‌شود. موفقیت بعضی قطعه‌ها می‌تواند نتیجه‌ای جزئی تولید کند، بدون آنکه fallback یا پرچم «قطعه‌ی حذف‌شده» فعال شود. شرط فعلی، خالی بودن فهرست را می‌سنجد و رشته‌ی خالیِ افزوده‌شده به آن را جدا کنترل نمی‌کند.

مسیر Google تفکیک گوینده ندارد. با diarize=True فقط warning ثبت و متن ساده برگردانده می‌شود. برای proxy، urllib opener به‌صورت عمومی نصب و در finally با None بازگردانده می‌شود. این تغییر در سطح فرایند است و لزوماً opener قبلی را حفظ نمی‌کند؛ هم‌زمانی درخواست‌ها نیز می‌تواند اثر بپذیرد. برای کاربرد جدی بهتر است transport خصوصی و قابل آزمون برای هر client وجود داشته باشد.

۸-۵. ارسال ورودی صوت به OpenRouter

OpenRouterSTTProvider کلید اختصاصی یا AI_API_KEY و مدل STT_OPENROUTER_MODEL می‌خواهد. فایل کامل با ffmpeg به WAV شانزده کیلوهرتز mono تبدیل و به Base64 تبدیل می‌شود. پیام user دو بخش text و input_audio دارد. prompt می‌خواهد گفتار دقیقاً رونویسی شود و فقط متن بازگردد. این مسیر از complete متنی جداست تا قرارداد LLMProvider تغییر نکند.

تبدیل تمام صوت به یک payload باعث می‌شود اندازه‌ی Base64 بیشتر از بایت اولیه باشد و همه‌ی مدل‌های کاتالوگ قابلیت audio نداشته باشند. کد chunking خاصی برای این provider ندارد. ظرفیت مدل منتخب، حد درخواست و زمان conversion باید مستقل سنجیده شوند. نام STT_OPENROUTER_MODEL در تنظیمات خالی است و تنظیم شدن AI_MODEL به‌تنهایی جای آن را پر نمی‌کند.

این provider مقدار proxy و timeout را در constructor می‌خواند، اما تماس از client_ عمومی STT انجام می‌شود؛ آن client با timeout شصت ثانیه ساخته شده و AI_PROXY را دریافت نمی‌کند. بنابراین وجود self.proxy اثبات اعمال proxy نیست. این شکاف از مسیر واقعی فراخوانی مشاهده شده و در گزارش به‌عنوان محدودیت کنونی ثبت می‌شود. مسیر OpenRouter صوت نیز diarization ندارد و با درخواست آن warning می‌دهد.

۸-۶. زنجیره‌ی جایگزین و معنای موفقیت

در نبود STT_FALLBACK_PROVIDERS، خطای provider مستقیم عبور می‌کند. در وجود فهرست، provider اصلی و سپس نام‌های جایگزین به ترتیب امتحان می‌شوند. SttUnavailableError، TranscriptionError و حتی exception عمومی ثبت و مسیر بعدی انتخاب می‌شود. نخستین نتیجه‌ی موفق بازگردانده می‌شود و شکست همه‌ی مسیرها یک خطای ترکیبی فارسی دارد. نام تکراری در تنظیمات حذف نمی‌شود و می‌تواند تماس دوباره بسازد.

fallback راهی برای ادامه‌ی سرویس است، نه تضمین حفظ توانایی‌های مسیر اصلی. اگر Speechmatics با تفکیک گوینده شکست بخورد و Google موفق شود، متن بدون تفکیک خواهد بود. همچنین مسیر اصلی می‌تواند متن خالی بازگرداند و facade آن را موفق بداند؛ fallback بر exception فعال می‌شود، نه بر ارزیابی کیفیت محتوا. بهتر است قرارداد آینده علاوه بر text، قابلیت‌ها و کامل بودن نتیجه را برگرداند.

۸-۷. فایل موقت و نکته‌ی Windows

descriptor _mkstemp_path ساخته‌شده را بلافاصله می‌بندد، زیرا Windows حذف فایل باز را محدود می‌کند. فایل منبع و WAVها در finally با cleanup_ پاک می‌شوند و OSError در پاک‌سازی best-effort گرفته می‌شود. مسیر نام فایل با suffix اصلی ساخته می‌شود و فرمان ffmpeg به‌صورت فهرست آرگومان اجرا می‌گردد؛ shell interpolation برای نام فایل استفاده نشده است.

در Google، فایل قطعه پس از تبدیل موفق به فهرست chunks اضافه می‌شود. اگر conversion پیش از افزودن آن شکست بخورد، finally فقط source و chunks ثبت‌شده را پاک می‌کند و فایل ساخته‌شده‌ی تازه ممکن است باقی بماند. آزمون موجود پاک‌سازی مسیر موفق را بررسی می‌کند؛ این شاخه‌ی شکست به آزمون جدا نیاز دارد. subprocessها timeout صریح ندارند و ffprobe یا ffmpeg معیوب می‌تواند زمان کار را از کنترل معمول STT خارج کند.

Dockerfile فعلی Backend، Tesseract را نصب می‌کند ولی ffmpeg و ffprobe را ندارد. اضافه شدن SpeechRecognition در requirements به معنای وجود ابزارهای سیستم در container نیست. بنابراین مسیرهای جدید Google و OpenRouter صوت در تصویر فعلی به آماده‌سازی اضافی نیاز دارند. این محدودیت از تفاوت وابستگی Python و executable ناشی می‌شود و پیش از ادعای قابلیت استقرار باید رفع و آزموده شود.

۸-۸. پاک‌سازی اختیاری و تأیید یادداشت

پس از ثبت رونویسی، attempt_cleanup از AI پیش‌فرض استفاده می‌کند. prompt فقط قالب‌بندی و روشن کردن همان گفته‌ها را می‌خواهد و افزودن تشخیص، دارو، علامت یا اندازه‌گیری را ممنوع می‌کند. transcript تا بیست هزار نویسه محدود می‌شود. نبود provider پاک‌سازی را رد می‌کند و وضعیت می‌تواند

pending بماند؛ شکست دیگر cleaned_note را خالی و cleanup_status را failed می‌کند. متن خام نباید جایگزین شود.

attempt_cleanup خطاهای اجرای complete و ثبت نتیجه را می‌گیرد، اما ساخت provider پیش از آن بلوک انجام می‌شود و در آن محل فقط AIUnavailableError گرفته شده است. exception دیگری از ساخت provider می‌تواند به process_ برسد؛ شاخه‌ی عمومی شکست در آن تابع transcript را خالی و transcription_status را failed می‌کند. برای حفظ قطعی رونویسی موفق، آزمون best-effort باید این شاخه را هم پوشش دهد.

مسیر /voice-notes/{voice_note_id}/note عنوان و content بازبینی‌شده را از کاربر می‌گیرد و ClinicalNote با source=voice و voice_note_id می‌سازد. این مرحله عمل صریح پزشک است. route شرطی برای completed بودن transcript یا برابر بودن متن با آن ندارد؛ می‌تواند متن ارسال‌شده‌ی کاربر را برای ضبط موجود ثبت کند. تأیید انسانی در قرارداد UI معنا دارد و باید با شناسه و زمان تأیید در سیاست آینده دقیق‌تر ثبت شود.

۹.۸- آزمون و سنجش کیفیت

در اجرای منتخب یازده آزمون STT پایه و بیست‌ویک آزمون Google/OpenRouter موفق شدند. آزمون‌ها conversion، recognize و HTTP خارجی را در مرز ماژول جایگزین می‌کنند. chunking، retry، متن غیرقابل تشخیص، قالب input_audio، نبود کلید و مدل، پاک‌سازی موفق و عبور fallback از این راه بررسی شده‌اند. این نتیجه به صدای واقعی، کیفیت فارسی، دقت نسبت دادن گوینده یا دسترس‌پذیری سرویس تعمیم داده نمی‌شود.

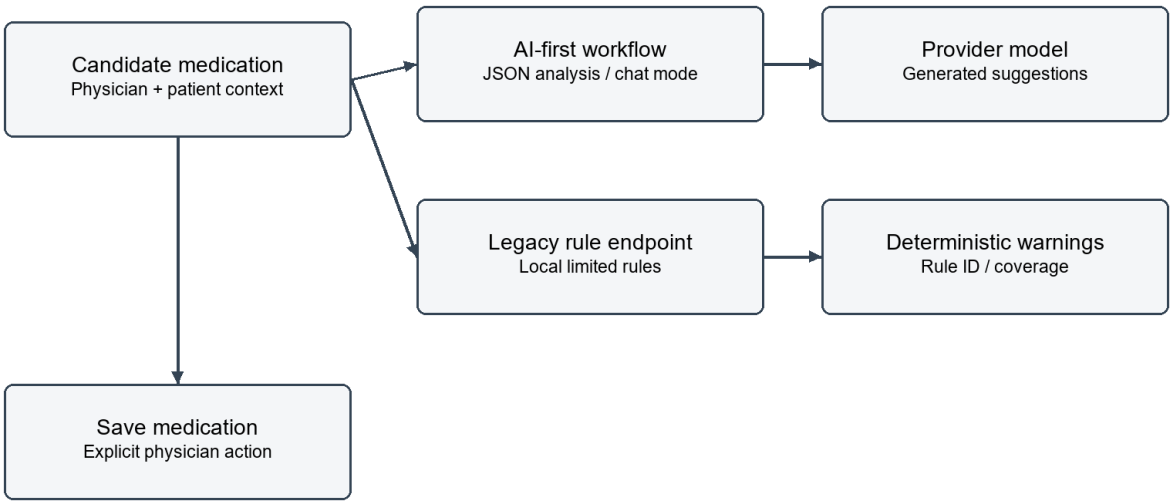
برای محاسبه‌ی WER باید تعداد جانشینی، حذف و درج بر تعداد واژه‌های متن مرجع تقسیم شود. CER همان منطق را روی نویسه دارد. در فارسی قواعد عادی‌سازی فاصله، نیم‌فاصله، شکل حروف و اعداد باید پیش از محاسبه مشخص شوند؛ وگرنه تفاوت نوشتاری با خطای گفتار مخلوط می‌شود. پروپوزال این ارزیابی را پیش‌بینی کرده، اما مجموعه‌ی مرجع و نتیجه‌ی عددی آن در شواهد بررسی‌شده وجود ندارد.

فصل نهم ایمنی دارو شواهد و عامل آزمایشی

۹-۱. تغییر جریان ایمنی نسبت به طرح اولیه

پروپوزال برای Prescription Safety بر هشدارهای ساخته‌شده با قواعد و نقش توضیح‌دهنده‌ی LLM تأکید دارد. این اصل ابتدا در services/medication_safety.py و مسیر /medications/check پیاده شده است. سپس سرویس ai_medication_safety و حالت ایمنی چت افزوده شده‌اند که از قواعد قدیمی استفاده نمی‌کنند. رابط افزودن دارو از endpoint AI و چت از prompt تخصصی استفاده می‌کند. بنابراین مسیر فعلی را باید AI-first نامید و تفاوت آن با اصل اولیه را آشکار کرد.

موتور قدیمی حذف نشده و router آن همچنان ثبت است. وجود هم‌زمان دو مسیر، حفاظت ترکیبی خودکار ایجاد نمی‌کند. در درخواست AI، نتیجه‌ی قواعد به prompt افزوده نمی‌شود و خروجی AI نیز برای هشدار قطعی معتبرسازی بالینی نمی‌شود. شکل ۹-۱ دو مسیر مستقل را نشان می‌دهد. فلش جداگانه‌ی ثبت دارو بیان می‌کند که هیچ‌یک از تحلیل‌ها شرط اجباری ذخیره نیست.



شکل ۹-۱ تفکیک تحلیل AI و موتور محدود قواعد از ثبت دارو

ارزش مسیر AI در امکان بیان ملاحظات متنوع و اطلاعات ناقص است، اما هزینه‌ی آن وابستگی به مدل، تغییرپذیری پاسخ و نبود پوشش قابل شمارش قواعد است. موتور محدود قواعد تکرارپذیرتر است، ولی نام ناشناخته و شرایط خارج از دانش محلی را نمی‌پوشاند. هیچ‌کدام از این ویژگی‌ها به‌تنهایی برتری بالینی را ثابت نمی‌کند. انتخاب مسیر باید با سناریوهای مرجع و توجه به خطای منفی کاذب ارزیابی شود.

۹-۲. قرارداد تحلیل ساختاریافته

AiMedicationSafetyCheckRequest یک تا پنج MedicationCandidate با name و dose اختیاری و question اختیاری دارد. Context کامل builder و سؤال ساخته‌شده‌ی build_safety_question به مدل پیش‌فرض می‌روند. پاسخ شامل overall_assessment، interactions، patient_factors، missing_information و uncertainty است و route شناسه‌ی بیمار، echo کاندیدها، generated_at و disclaimer ثابت را اضافه می‌کند. داده‌ی این تحلیل در جدول مستقلی ذخیره نمی‌شود.

parse_analysis ابتدا JSON میان نخستین و آخرین آکولاد را پیدا می‌کند و fence را کنار می‌گذارد. overall_assessment باید رشته‌ی غیرخالی باشد و uncertainty رشته یا .null فیلدهای فهرستی باید list باشند و اعضای غیررشته‌ای حذف می‌شوند. نبود برخی فهرست‌ها به فهرست خالی تبدیل می‌شود. متن آزاد یا نوع نامعتبر به AIError و سپس ۵۰۲ می‌انجامد. بنابراین raw output شکسته به‌عنوان تحلیل ساختاریافته عرضه نمی‌شود.

این کنترل‌ها شکل داده را تضمین می‌کنند، نه صحت محتوا. prompt از مدل می‌خواهد تداخل احتمالی و شناخته‌شده را جدا و اطلاعات ناقص را بیان کند. parser وجود پیشوند درست هر interaction یا پشتوانه‌ی داروشناختی آن را راستی‌آزمایی نمی‌کند. schema شدت، rule_id یا شناسه‌ی استاندارد دارو ندارد. نداشتن هشدار در این خروجی نباید به معنی تجویز بی‌خطر تلقی شود.

۹-۳. حالت ایمنی در چت

فلگ medication_safety در چت معمول و ماندگار prompt را تغییر می‌دهد. تشخیص واژه‌ی دارو یا انتخاب خودکار حالت در کد وجود ندارد؛ کاربر toggle را فعال می‌کند. در مسیر ماندگار، قواعد evidence با prompt ایمنی ترکیب می‌شوند و پاسخ JSON شامل answer و evidence انتظار می‌رود. endpoint اختصاصی تحلیل، قرارداد متفاوتی با فهرست تعامل و عوامل بیمار دارد. یکسان بودن موضوع، یکسان بودن schema را ایجاب نمی‌کند.

تحلیل شکست‌خورده ثبت دارو را مسدود نمی‌کند و interface می‌تواند خطا را جدا نشان دهد. این جداسازی از وابستگی عملکرد پایه به سرویس بیرونی جلوگیری می‌کند. با این حال نباید به کاربر علامت موفق ایمنی بدهد وقتی فقط ذخیره موفق شده است. نتیجه‌ی تحلیل، وضعیت اجرا و عمل کاربر باید در ارائه‌ی آینده روشن و برای ممیزی قابل بازیابی باشند.

۹-۴. موتور قواعد محدود

medication_rules.json کلاس دارو، alias، interaction و cross_reactivity را تعریف می‌کند. load_rules_kb فایل را یک بار خوانده و dataclassهای ثابت می‌سازد. normalize_drug_name حروف عربی ی و ک را به فارسی تبدیل می‌کند، فاصله و نیم‌فاصله و نشانه‌ها را عادی می‌سازد و اعداد دوز و بعضی

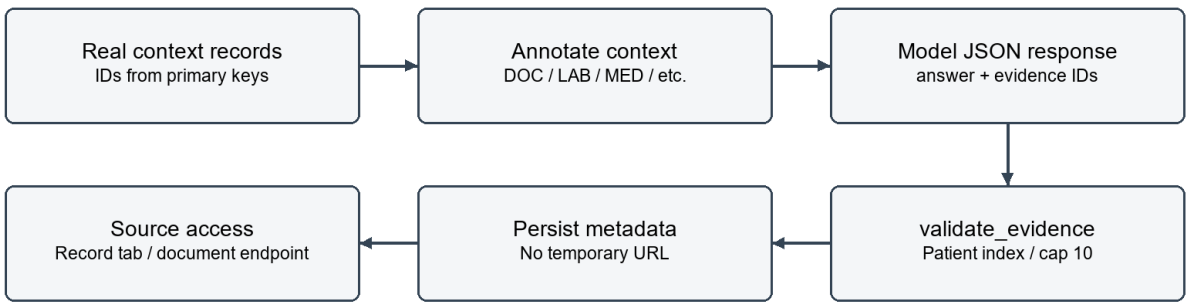
واحدها را کنار می‌گذارد. match_classes با alias به کلاس نگاشت می‌کند؛ این روش شناسه‌ی استاندارد دارویی یا حل جامع نام تجاری نیست.

check_medication_safety کاندید را با داروهای active و allergenهای پرونده می‌سنجد. هشدارها از سه نوع drug_interaction، allergy_conflict و duplicate_therapy هستند. هر هشدار rule_id، severity، title، detail و involved دارد و بر شدت مرتب می‌شود. برای کلاس دارای duplicate_allowed هشدار تکرار درمان ایجاد نمی‌شود. نامی که به کلاس شناخته‌شده نگاشت نشود، هشدار تازه نمی‌سازد و candidate_covered=false وضعیت پوشش را بیان می‌کند.

مسیر /medications/check می‌تواند کاندید را با medication_id یا medication_name بگیرد و در صورت explain، مدل فقط هشدارهای قطعی تولیدشده را توضیح می‌دهد. شکست توضیح به explanation=null می‌انجامد و هشدارها باقی می‌مانند. داده‌ی محلی قواعد را نباید دانش‌نامه‌ی تأییدشده‌ی پزشکی نامید؛ وجود rule_id به معنای نسخه‌پذیری محلی است و نه اعتبار بالینی همه‌ی متن قواعد.

۹-۵. شواهد پرونده و شناسه‌ی پایدار

build_evidence_index برای هر رکورد موجود در Context یک ID با پیشوند گروه و کلید اصلی می‌سازد. MED، ALLERGY، COND، LAB، DOC، NOTE و VOICE پیشوندهای واقعی‌اند. annotate_context این شناسه را کنار رکورد به پیام مدل اضافه می‌کند. مدل اجازه دارد فقط همین شناسه‌ها را استفاده کند و نباید مقاله یا URL بسازد. شکل ۹-۲ مسیر اعتبارسنجی و پیوند منبع را نشان می‌دهد.



شکل ۹-۲ اعتبارسنجی شناسه‌ی شواهد و دسترسی به منبع

validate_evidence شناسه‌های تکراری را کنار می‌گذارد و فقط موارد موجود در index همان بیمار را نگه می‌دارد؛ سقف پاسخ ده شاهد است. metadata از رکورد اصلی ساخته می‌شود و document_id تنها در Document یا رابطه‌ی واقعی آزمایش با سند وجود دارد. نشانی دانلود موقت در شواهد ذخیره نمی‌شود. این روش مانع تبدیل ID جعلی یا متعلق به بیمار دیگر به منبع معتبر در فهرست ساختاریافته می‌شود.

اعتبار ID با صحت استناد فرق دارد. مدل ممکن است MED واقعی را ذکر کند ولی از آن نتیجه‌ی پزشکی پشتیبانی‌نشده بگیرد. validate_evidence تطابق معنایی جمله و منبع را بررسی نمی‌کند و محتوای answer

را نیز برای شناسه‌های جعلی داخل متن بازنویسی نمی‌کند. بنابراین فهرست شواهد کنترل‌شده است، اما کل پاسخ با verifier معنایی تأیید نشده است. این مرز باید در هر ارزیابی Citation Accuracy رعایت شود.

۹-۶. تجزیه‌ی پاسخ و نمایش منبع

parse_evidence_answer JSON با answer رشته‌ای و evidence فهرستی را می‌پذیرد. اگر پاسخ نثر یا JSON نامعتبر باشد، متن خام به‌عنوان answer و evidence خالی برمی‌گردد. این fallback پاسخ قابل خواندن را حفظ می‌کند، ولی عدم رعایت قرارداد JSON را به خطای قطعی تبدیل نمی‌کند. این رفتار با parser تحلیل دارو که در خطای ساختار ۵۰۲ می‌دهد متفاوت است و باید به مقصد مناسب نسبت داده شود.

component EvidenceList نوع منبع و شرح خوانا را نمایش می‌دهد؛ منابع دارای document_id به دانلود اصل سند و سایر منابع به تب پرونده هدایت می‌شوند. label دارو در UI «داروی فعال» است، در حالی که Context می‌تواند داروی stopped یا completed هم داشته باشد. این برچسب برای تمام شواهد دارویی دقیق نیست. در توسعه‌ی بعدی نمایش status در metadata و label وابسته به آن از برداشت اشتباه جلوگیری می‌کند.

شواهد پرونده فقط واقعیت ثبت‌شده درباره‌ی بیمار را پشتیبانی می‌کنند. رکورد مصرف یک دارو، مرجع اثبات تداخل عمومی آن دارو نیست. کد prompt همین تمایز را بیان می‌کند. PubMed، FDA و RxNorm در طرح به‌عنوان منابع ممکن آمده‌اند، اما سرویس بازیابی خارجی یا اتصال عملیاتی آن‌ها در جریان فعلی وجود ندارد. فهرست منابع علمی گزارش نیز نباید با منابعی که سامانه هنگام پاسخ خوانده مخلوط شود.

۹-۷. ابزارهای محلی MCP و عامل آزمایشی

MCP، مخفف Model Context Protocol، در این مخزن یک رابط ابزار محلی و فقط‌خواندنی است. app.mcp_server شش ابزار برای profile، Context، labs، medications، metadata اسناد و timeline دارد. هر ابزار Session خود را می‌سازد و داده‌ی بیمار را از مدل‌ها و builder موجود می‌خواند. ورودی، پارامترهای عددی مشخص است؛ SQL، نام جدول یا storage_key دلخواه از مدل گرفته نمی‌شود.

MCPStdioClient یک subprocess با همان Python و python -m app.mcp_server راه‌اندازی می‌کند. ارتباط stdio روی event loop پس‌زمینه و با thread انجام می‌شود تا route همگام از client غیرهمگام استفاده کند. ساخت session سی ثانیه و call ابزار شصت ثانیه timeout دارد. session cache می‌شود و در خرابی reset انجام می‌گیرد. این مسیر یک آزمایش معماری است و تحمل خطا و lifecycle کامل سرویس عملیاتی برای آن اثبات نشده است.

run_agent نخست از مدل برنامه‌ی ابزار را به JSON می‌خواهد. نام‌ها با TOOL_LABELS محدود و تکراری‌ها حذف می‌شوند و حداکثر سه ابزار اجرا می‌شود. شناسه‌ی بیمار را Backend از patient مجوزدار تزریق می‌کند؛ مدل فقط نام ابزار می‌گوید. نتیجه‌ی هر ابزار تا چهار هزار نویسه محدود و در فراخوانی دوم به

مدل داده می‌شود. مسیر answer مستقیم برای سؤال عمومی نیز وجود دارد و کنترل مستقل برای عمومی بودن سؤال اعمال نمی‌شود.

۹-۸. امنیت و محدودیت آزمایش عامل

endpoint عامل از patient_from_id استفاده می‌کند، ولی MCP مستقیم هویت کاربر یا ACL وب را دریافت نمی‌کند. بنابراین فقط وقتی subprocess در مرز اعتماد Backend باقی بماند می‌توان به بررسی دسترسی endpoint تکیه کرد. ارائه‌ی مستقیم ابزار به کاربر یا شبکه به سازوکار مجوز جدا نیاز دارد. read-only بودن مانع خواندن غیرمجاز داده‌ی محرمانه نمی‌شود.

دامنه‌ی داده‌ی ابزارها یکسان نیست. get_patient_documents فقط متادیتای انتخاب‌شده می‌دهد و متن و storage_key را حذف می‌کند؛ get_patient_context خروجی کامل builder، شامل متن استخراج‌شده، را بازمی‌گرداند. بنابراین محدودیت متن فقط به ابزار متادیتای اسناد مربوط است. test_mcp_tools شکل این خروجی‌ها را تعریف می‌کند؛ اجرای زنده‌ی subprocess در بررسی حاضر انجام نشده است.

عامل stateless است و پاسخ آن در Conversation ذخیره نمی‌شود. شکست MCP یا AI به خطای تمیز می‌انجامد و مسیر چت معمول جدا باقی می‌ماند. مسیر agent ابزار نسخه‌نویسی یا تغییر پرونده ندارد. این جدایی ارزش نمونه‌ی آزمایشی را نشان می‌دهد، اما نباید به‌عنوان عامل بالینی پایدار یا مستقل معرفی شود. ثبت ابزار استفاده‌شده شفافیت فنی ایجاد می‌کند و جای ارزیابی معنایی پاسخ را نمی‌گیرد.

فصل دهم احراز هویت امنیت و حریم خصوصی

۱۰-۱. حساب کاربری و توکن

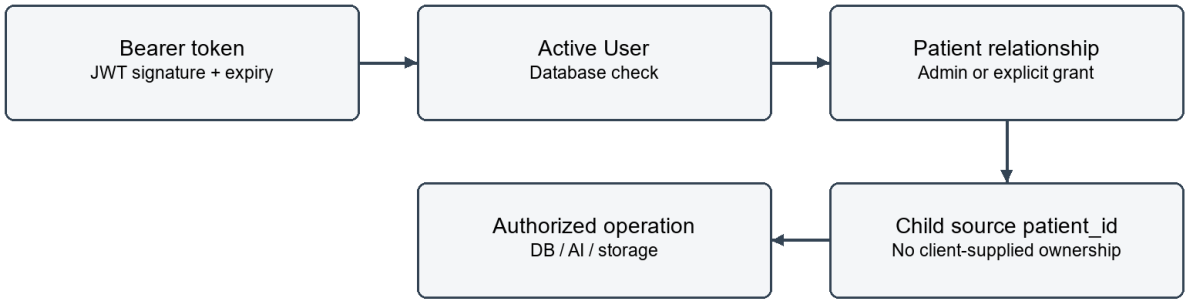
ثبت‌نام ایمیل و گذرواژه حساب doctor می‌سازد و درخواست نقش admin رد می‌شود. ایمیل به lowercase تبدیل و password با bcrypt ذخیره می‌شود. پاسخ UserResponse hash را عرضه نمی‌کند. login خطای ایمیل ناشناخته و گذرواژه‌ی غلط را با پیام یکسان بیان می‌کند و کاربر غیرفعال نیز توکن نمی‌گیرد. نقش admin از مسیر seed توسعه تعریف می‌شود و یک فرایند تأیید رسمی صلاحیت پزشکی در ثبت‌نام وجود ندارد.

JWT، مخفف JSON Web Token، شامل sub برابر id کاربر و exp است. algorithm پیش‌فرض HS256 و عمر پیش‌فرض چهارصدوهشتاد دقیقه است. decode فقط الگوریتم تنظیم‌شده را می‌پذیرد و subject را به عدد تبدیل می‌کند. get_current_user سپس User را از پایگاه می‌خواند و is_active را می‌سنجد. توکن فقط ادعای امضاشده است؛ فعال بودن حساب و مجوز پرونده در پایگاه دوباره کنترل می‌شوند.

JWT_SECRET_KEY در Settings یک مقدار توسعه‌ی ناامن دارد و باید پیش از هر استقرار غیرآزمایشی جایگزین شود. گزارش مقدار واقعی محیط یا کلید را بازنشر نمی‌کند. وجود متغیر محیطی به معنی مدیریت حرفه‌ای secret، rotation یا جلوگیری از همه‌ی نشت‌ها نیست. refresh token، فهرست ابطال token و نشست دستگاه در پیاده‌سازی فعلی دیده نمی‌شود؛ خروج از رابط توکن محلی را پاک می‌کند، ولی لزوماً نسخه‌ی کپی‌شده‌ی آن را باطل نمی‌کند.

۱۰-۲. اجازه‌ی پرونده و منابع فرزند

user_has_patient_access برای admin true می‌دهد و برای doctor وجود زوج PatientUserAccess را می‌سنجد. patient_from_id بعد از یافتن بیمار ensure_patient_access را اجرا می‌کند. در منبع مستقل مانند Document یا Conversation، helper رکورد را می‌خواند و patient_id واقعی آن را برای کنترل استفاده می‌کند. این الگو مانع تکیه بر patient_id ادعایی مرورگر برای مجوز یک سند می‌شود. شکل ۱۰-۱ جریان مجوز را نمایش می‌دهد.



شکل ۱۰-۱ احراز هویت و کنترل انتساب بیمار

برگرداندن ۴۰۴ برای منبع ناموجود و ۴۰۳ برای منبع موجود ولی غیرمجاز می‌تواند تفاوت وجود شناسه را نشان دهد. این رفتار فعلی است و سیاست پنهان‌سازی وجود منابع در همه‌ی routeها یکسان‌سازی نشده است. همچنین نقش doctor دارای دسترسی، سطح read-only و write جدا ندارد؛ قرارداد ACL فعلی انتساب پرونده را کنترل می‌کند، نه تمام سطح‌های مجوز سازمانی.

test_patient_access_control.py دسترسی doctor به پرونده‌ی خود و دیگری، bypass ادمین و کنترل مسیرهای مستقل را بررسی می‌کند. تست دانلود ترتیب مجوز پیش از presign را می‌سنجد و تست agent اجازه را پیش از MCP بررسی می‌کند. این آزمون‌ها به پایگاه آزمایشی وابسته‌اند و در اجرای محدود تازه اجرا نشده‌اند. وجود آن‌ها شواهد طراحی آزمون است و نتایج تاریخی در docs باید با تاریخ خود خوانده شوند.

۱۰-۳. ورود یک‌بارمصرف

ورود شماره‌ی تلفن در /auth/request-otp و /auth/verify-otp تعریف شده است. normalize_phone شکل‌های معمول شماره‌ی ایرانی را به E.164 تبدیل می‌کند. issue_otp با secrets.randbelow کد شش‌رقمی می‌سازد و hash SHA-256 از phone و code نگه می‌دارد. زمان انقضا ۱۲۰ ثانیه و attempts_left پنج است. verify جدیدترین درخواست را می‌خواند و پس از مقایسه‌ی hash با hmac.compare_digest، کد را مصرف می‌کند.

این hash مانع نگهداری متن ساده‌ی کد در جدول است، اما فضای شش‌رقمی محدود است و hash بدون secret در برابر مهاجم دارای پایگاه، مانع قوی حدس آفلاین نیست. طرح محدودیت درخواست، حداکثر سه کد در ده دقیقه را هدف می‌گیرد. شرط فعلی recent[0].created_at را با window_start می‌سنجد؛ اگر سه رکورد موجود باشند، تازه بودن جدیدترین شرط رد است حتی اگر دو رکورد دیگر خارج بازه باشند. بنابراین رفتار دقیق می‌تواند سخت‌گیرانه‌تر از شمارش سه درخواست در پنجره باشد.

کد یک‌بارمصرف از طریق Uniom در پیام‌رسان Soroush Plus ارسال می‌شود. در نبود تنظیمات، route پاسخ ۵۰۳ می‌دهد و جدیدترین رکورد همان شماره را حذف می‌کند؛ تحت رقابت، این رکورد الزاماً همان

درخواست فراخواننده نیست. شکست ارسال UniomError به ۵۰۲ تبدیل می‌شود و شماره‌ی مقصد در log خطا ثبت می‌گردد. این رفتار با توضیح ابتدای سرویس مبنی بر ثبت نشدن شماره در log سازگار نیست.

verify-otp کاربر متناظر را پیدا یا می‌سازد و همان JWT سیستم را می‌دهد. مصرف کد و ساخت حساب مراحل پایگاه‌اند و مدیریت هم‌زمانی دو درخواست مصرف باید به‌طور جدا بررسی شود. lock یا update شرطی اتمیک برای جلوگیری از race در verify دیده نمی‌شود. تست‌های معمول انقضا و مصرف دوباره مفیدند، اما هم‌زمانی نیازمند سناریوی مستقل است.

۱۰-۴. نگهداری توکن در رابط

auth.ts توکن را در localStorage و cookie access_token نگه می‌دارد. /=cookie path، عمر هشت ساعت و SameSite=Lax دارد، ولی HttpOnly و Secure در رشته‌ی ساخته‌شده نیستند. این انتخاب برای انتقال توکن به Server Component ساده است؛ هزینه‌ی آن قابل خواندن بودن توکن با JavaScript است. در صورت آسیب‌پذیری XSS، مهاجم می‌تواند آن را بردارد. توصیه‌ی امنیتی مربوط به این نوع ذخیره را باید با منبع تخصصی و مدل تهدید بررسی کرد [۱۰].

Backend header-only است و cookie را مستقیم نمی‌خواند. درخواست API از مرورگر Bearer اضافه می‌کند و درخواست سروری آن را از cookie منتقل می‌کند. این تفاوت علت شکاف پیوند مستقیم دانلود است. استفاده‌ی cookie برای SSR به‌تنهایی یک راهکار کامل session محسوب نمی‌شود. توسعه‌ی بعدی می‌تواند واسط سروری با cookie امن و proxy دانلود مجوزدار را بررسی کند و هم‌زمان CSRF و خروج و انقضا را آزمون کند.

۱۰-۵. نگهداری فایل، داده و گزارش‌های اجرا

کلید ذخیره UUID دارد، ولی فایل‌ها در حالت معمول با MinIO نگهداری می‌شوند و رمزگذاری در سطح برنامه برای متن یا فایل در کد بررسی‌شده وجود ندارد. MINIO_SECURE انتخاب transport است و به معنی رمزگذاری پایگاه یا دیسک نیست. فایل موقت صوت نیز روی دیسک ایجاد می‌شود؛ پاک‌سازی معمول انجام می‌گیرد، اما خرابی فرایند می‌تواند آن را باقی بگذارد. retention و پاک‌سازی پس از crash باید جزء طراحی عملیاتی باشند.

AI context مشخصات بیمار را با متن و رکوردها به سرویس بیرونی می‌فرستد. کد رضایت، ناشناس‌سازی خودکار یا قرارداد منطقه‌ی پردازش را پیاده نکرده است. برای داده‌ی واقعی باید حداقل‌سازی داده، ضرورت ارسال و شرایط نگهداری سرویس بررسی شوند. این گزارش داده‌ی بیمار واقعی را نمونه قرار نمی‌دهد و مثال‌های پیوست مصنوعی‌اند. داشتن نمونه‌ی آموزشی مجوز کاربرد عملیاتی در بیمارستان نیست.

در logging، بخشی از پاسخ AI و در خطای چت بخشی از سؤال و در خطای OTP شماره دیده می‌شود. ClinicalEvent هم همه‌ی خواندن‌ها، تغییرها و هویت انجام‌دهنده را ثبت نمی‌کند. بنابراین لاگ موجود یک

audit trail جامع بالینی نیست. افزودن ثبت حداقل عملیات حساس با redaction و سیاست دسترسی، از چاپ متن آزاد در log مفیدتر است و باید با نیاز عیب‌یابی متوازن شود.

۱۰-۶. کنترل‌های موجود و کارهای باقی‌مانده

CORS مبداهای مشخص را می‌پذیرد و preflight برای مبدا نامجاز رد می‌شود. این کنترل رفتار مرورگر را محدود می‌کند، نه client مستقل را؛ ابزار خارج مرورگر می‌تواند درخواست مستقیم بفرستد و همچنان به احراز هویت نیاز دارد. بررسی MIME و نام فایل نیز مفید است، ولی ضدبدافزار، اسکن محتوای فعال و sandbox parser مستقل نیست. هر کنترل باید در محدوده‌ی واقعی‌اش توصیف شود.

اولویت امنیتی آینده شامل secretهای غیرتوسعه، TLS واقعی، cookie امن، کنترل دانلود رابط، حذف داده‌ی حساس از log، محدودیت نرخ برای login، سیاست نگهداری فایل و آزمون race است. این موارد از شواهد کد و مسیرهای مشاهده‌شده نتیجه می‌شوند؛ هیچ ادعای انطباق قانونی یا گواهی امنیتی از وجود bcrypt، JWT و ACL استنتاج نمی‌شود. تأیید بالینی و مقرراتی فرایندی مستقل از آزمون نرم‌افزار است.

فصل یازدهم آزمون یکپارچه‌سازی و استقرار

۱۱-۱. ساختار آزمون‌ها

مخزن تست‌های service، schema، ORM و API و چند script سرتاسری دارد. conftest یک TestClient بدون پایگاه و fixtureهای پایگاه واقعی تعریف می‌کند. نام پایگاه تست با پسوند test_ از تنظیمات گرفته می‌شود. db_engine می‌تواند آن پایگاه را بسازد، metadata.create_all اجرا کند و در پایان drop_all انجام دهد. این فرایند روی پایگاه تست است، اما پیش از اجرای کامل باید اختصاصی بودن آن تأیید شود؛ اجرای بی‌بررسی روی محیط مشترک مناسب نیست.

db_session با تراکنش اتصال و create_savepoint کار می‌کند تا commitهای route در تست قابل بازگردانی باشند. api_client کاربر admin می‌سازد و get_db و get_current_user را override می‌کند. تست‌های اختصاصی auth و access client دیگری با وابستگی واقعی می‌سازند. تفکیک این دو گروه لازم است؛ سبز بودن تست CRUD با admin جای بررسی مجوز doctor را نمی‌گیرد.

در تست providerها، MockTransport پاسخ شبکه را کنترل می‌کند. در تست STT جدید، ffprobe، ffmpeg و recognize در مرز ماژول stub می‌شوند. این روش تست را از هزینه و ناپایداری سرویس بیرونی جدا می‌کند و امکان بررسی شاخه‌ی ۴۲۹ یا پاسخ نامعتبر را می‌دهد. در مقابل کیفیت سرویس واقعی و سازگاری فایل کامل باید با آزمون دیگری سنجیده شوند.

۱۱-۲. بازاجرای آزمون‌های منتخب

در بازبینی ۱۴ اکتبر ۲۰۲۶، همان هشت فایل health، CORS، logging، storage، extraction، STT پایه، STT جدید و OpenRouter provider دوباره اجرا شدند. Python 3.12.2 و pytest 8.4.2 استفاده شد؛ وابستگی pytest از زیرپوشه‌ی ابزار گزارش خوانده شد و requirements برنامه تغییر نکرد. ۶۱ مورد جمع‌آوری شد: ۵۹ موفق، صفر failed، دو error در setup و صفر .skipped این آمار از XML و خلاصه‌ی اجرای تازه خوانده شده است.

هر دو error در fixture فایل test_storage_service.py و پیش از اجرای بدنه‌ی آزمون رخ دادند: اتصال به localhost:9000 رد شد. fixture فقط StorageError را به skip تبدیل می‌کند، ولی خطای اتصال مشاهده‌شده از این شاخه عبور نکرد و error باقی ماند. هفت آزمون استخراج، یازده STT پایه، بیست‌ویک STT جدید و یازده OpenRouter، همراه health و CORS و logging موفق شدند. مدت خلاصه‌ی pytest، ۶۱٫۳۶ ثانیه بود؛ این مقدار زمان اجرای مجموعه‌ی تست است.

جدول ۶ نتیجه‌ی اجرای منتخب

فایل آزمون	موفق	خطا	کنارگذاشته
test_health	3	0	0
test_cors	4	0	0
test_logging	2	0	0
test_storage_service	0	2	0
test_extraction_service	7	0	0
test_stt_service	11	0	0
test_stt_google_and_openrouter	21	0	0
test_openrouter_provider	11	0	0

این مجموعه تماس‌های AI و STT را جایگزین می‌کند و PostgreSQL، Docker و گردش کار سرتاسری را اجرا نکرده است. اتصال واقعی OCR در آزمون استخراج بررسی شد و اتصال ذخیره‌ساز به خطای setup رسید. دامنه‌ی ۵۹ نتیجه‌ی موفق به قراردادهای همین فایل‌ها محدود است. فرمان و آرگومان‌های مؤثر اجرا در گزارش اعتبارسنجی ثبت شده‌اند؛ جدول فصل حاضر دامنه‌ی نتیجه‌ها را نشان می‌دهد.

۱۱-۳. نتایج تاریخی و تفسیر آن‌ها

docs/task-19-final-integration-e2e-report.md اجرای تاریخی ۳۶ مرحله‌ی smoke، آزمون دسترسی و بررسی migration را گزارش کرده است. همان سند تصریح می‌کند که تماس موفق واقعی Gemini به دلیل محدودیت شبکه روی محیط آن تسک تأیید نشده است. برخی عنوان‌های اسناد هنوز می‌گویند commit انجام نشده، در حالی که Git commit متناظر را دارد. تاریخچه‌ی نسخه‌ی فعلی نسبت به توضیح قدیمی سند برای وضعیت commit مقدم است.

مستند Task 14 نتیجه‌ی تست با provider جایگزین و مسیر خطای زنده‌ی AI را از هم جدا می‌کند. از این گزارش نمی‌توان کیفیت پزشکی تحلیل یا موفقیت حال حاضر Google را نتیجه گرفت. این اسناد برای فهم هدف تست و خطاهای شناخته‌شده مفیدند، اما بازاجرای مستقل نیستند. در ارزیابی نهایی بهتر است هر نتیجه با نسخه‌ی کد، تنظیم غیرمحرمانه و زمان اجرا همراه باشد.

۱۱-۴. طرح ارزیابی و معیارهای غایب

در شواهد بررسی‌شده WER، CER، P50، P95، SUS یا حساسیت بالینی با مجموعه‌ی مرجع گزارش نشده است. برای سنجش پاسخ AI باید دو حالت بدون زمینه و همراه زمینه بر سؤال‌های یکسان مقایسه شوند. ارزیاب باید علاوه بر مرتبط بودن پاسخ، ادعای unsupported، حفظ مقدار عددی، بیان اطلاعات ناقص و نسبت درست منبع را ثبت کند. این پیشنهاد، نتیجه‌ی انجام‌شده نیست و به‌عنوان طرح ارزیابی آینده ارائه می‌شود.

برای ایمنی دارو، true positive، false positive، true negative و false negative باید از سناریوی مرجع مستقل به دست آیند. حساسیت از TP تقسیم بر TP+FN و ویژگی از TN تقسیم بر TN+FP محاسبه می‌شود؛ کم بودن خطای منفی کاذب اهمیت ویژه دارد. نداشتن warning برای نام خارج پوشش نباید true negative شمرده شود. در مسیر AI، نتیجه‌ی عدم قطعیت نیز باید جدا از «ایمن» برچسب بخورد.

تأخیر باید برای مسیرهای جدا اندازه‌گیری شود: CRUD، upload تا ثبت، upload تا استخراج، سؤال تا پاسخ و صوت تا .transcript میانگین ساده می‌تواند دنباله‌ی بلند شبکه را پنهان کند؛ صدک‌ها و درصد timeout اطلاعات بیشتری می‌دهند. کنترل تعداد و طول درخواست، وضعیت cache و مدل لازم است. زمان اجرای pytest یا مقدار timeout در settings اندازه‌گیری latency واقعی نیست.

۱۱-۵. زیرساخت توسعه

docker-compose.yml ریشه فقط PostgreSQL و MinIO را اجرا می‌کند و Backend و Frontend در توسعه محلی‌اند. PostgreSQL از تصویر postgres:17-alpine و MinIO از latest استفاده می‌کند. هر دو volume دارند و healthcheck تعریف شده است. در این فایل پورت‌های پایگاه و ذخیره‌ساز به میزبان منتشر می‌شوند. این توپولوژی برای توسعه با فایل production متفاوت است و نباید شرح یکی جای دیگری بنشیند.

infra/README.md همان مرحله‌ی توسعه را شرح می‌دهد و می‌گوید container برنامه هنوز وجود ندارد. Dockerfileهای جدید و deploy/docker-compose.prod.yml بعداً به مخزن افزوده شده‌اند. این ناهماهنگی مستندات از مواردی است که بررسی کد آن را اصلاح می‌کند. گزارش استقرار، هر فایل را با دامنه‌ی خودش معرفی می‌کند و README قدیمی را وضعیت نهایی محصول نمی‌نامد.

۱۱-۶. تصویر Backend و Frontend

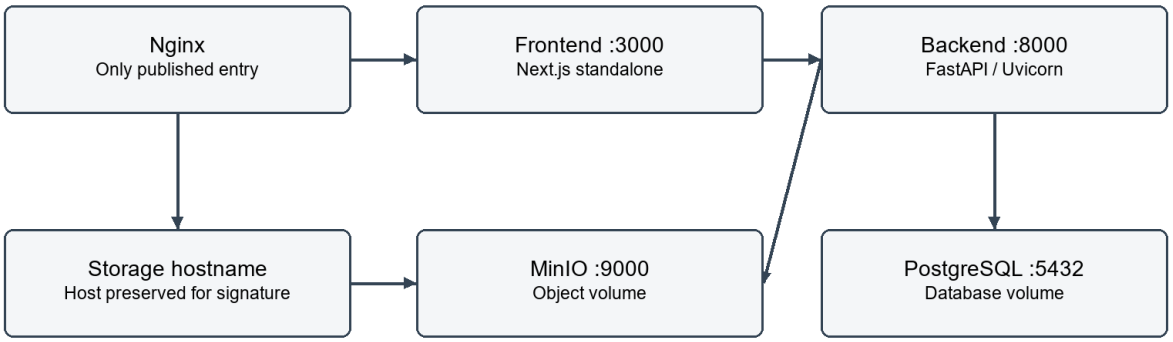
Backend بر python:3.12-slim بنا شده و Tesseract و زبان فارسی را نصب می‌کند. requirements پیش از کد کپی می‌شود تا cache لایه‌ی dependency استفاده شود. app، migrationها و scriptها وارد تصویر می‌شوند. کاربر appuser غیرroot با uid هزار برنامه را اجرا می‌کند و uvicorn روی 0.0.0.0:8000 قرار دارد. healthcheck از urllib درخواست health/ می‌زند. وجود این تنظیمات به معنی آزمون موفق build در اجرای حاضر نیست.

Frontend سه مرحله دارد: deps با npm ci از lockfile، builder با next build و runner با خروجی standalone. تصویر پایه node:24-alpine و runtime با کاربر node است. NEXT_PUBLIC_API_URL هنگام build در bundle قرار می‌گیرد و تغییر env پس از build لزوماً client را عوض نمی‌کند. مستندات Next.js درباره‌ی self-hosting و تنظیمات runtime توضیح می‌دهد [۳]؛ جزئیات تصویر حاضر از Dockerfile است.

نسخه‌های وابستگی Backend عمدتاً بازه‌ای‌اند و MinIO latest استفاده می‌شود. این انتخاب نگهداری را آسان‌تر می‌کند، ولی build دو زمان مختلف الزاماً بایت یکسان ندارد. برای تکرارپذیری می‌توان lock یا pin کنترل‌شده و ثبت digest تصویر را اضافه کرد. هر ارتقا نیز باید با تست سازگاری و migration بررسی شود، نه فقط نصب موفق .package

۱۱-۷. توپولوژی استقرار و ترتیب راه‌اندازی

پیکربندی production پنج سرویس nginx، frontend، backend، postgres و minio دارد. تنها nginx پورت ۸۰ و ۴۴۳ را publish می‌کند؛ سایر سرویس‌ها expose داخلی دارند. Backend منتظر healthy شدن پایگاه و MinIO است، ولی Frontend فقط Backend service_started را می‌خواهد. Nginx نیز dependency آغاز سرویس دارد. شکل ۱-۱۱ شبکه و volumeها را نمایش می‌دهد.



شکل ۱-۱۱ توپولوژی تعریف‌شده در Compose استقرار

healthcheck پایگاه با pg_isready و MinIO با mc ready است. Backend health فقط پاسخ HTTP را بررسی می‌کند. شرط started برابر آماده بودن تمام endpointها نیست و صحت migration را تضمین نمی‌کند. مستندات Docker تفاوت ترتیب راه‌اندازی و healthcheck را توضیح می‌دهد [۹]. در این مخزن migration عمداً خودکار نیست و باید به‌عنوان مرحله‌ی جدا اجرا شود.

۱۱-۸. Nginx و نشانی امضاشده

در Nginx، /api/ و /health به Backend و سایر مسیرهای برنامه به Frontend می‌روند. میزبان storage.local.test و wildcard متناظر به MinIO هدایت می‌شوند. Host بدون تغییر مهم است، زیرا امضای S3 به آن وابسته است. Backend با MINIO_ENDPOINT برابر hostname عمومی و extra_hosts نوع host-gateway به همان مسیر می‌رسد. اگر URL با minio:9000 ساخته شود مرورگر بیرون شبکه آن را نمی‌تواند resolve کند.

فایل پیکربندی فعلی فقط 80 listen دارد و comment نیز TLS را مرحله‌ی بعد می‌داند. publish کردن ۴۴۳ و mount کردن پوشه‌ی certs، TLS فعال ایجاد نمی‌کند. hostnameها نیز placeholder محیط

محلی‌اند و باید برای دامنه‌ی واقعی تغییر کنند. timeout proxy برای API سیصد ثانیه است؛ چند فراخوانی عامل یا retry ممکن است بودجه‌ی بیشتری بخواهد و باید در آزمون طولانی بررسی شود.

client_max_body_size در هر دو server برابر 25m است. از نگاه Backend صوت تا پنجاه MiB مجاز است، پس این اختلاف باید پیش از استقرار نهایی حل شود. volumeهای postgres_data و minio_data دوام داده پس از recreate container را هدف می‌گیرند، اما backup، restore و رمزگذاری را خودکار فراهم نمی‌کنند. وجود volume را نباید تضمین بازیابی پس از خرابی دیسک دانست.

۱۱-۹. رویه‌ی راه‌اندازی و وضعیت ارزیابی استقرار

رویه‌ی مستند شامل تنظیم محیط، build تصاویر، شروع data services، اجرای alembic upgrade head و بالا آوردن سرویس برنامه است. در محیط development می‌توان Backend و Frontend را محلی اجرا کرد؛ در production فایل Compose جدا استفاده می‌شود. تنظیمات منتخب در پیوست ب خلاصه شده‌اند و مقادیر secret حذف شده‌اند. seed صرفاً برای محیط آزمایشی و داده‌ی مصنوعی مناسب است و نباید بدون بررسی وارد محیط واقعی شود.

در این کار container ساخته یا روی سرور عمومی منتشر نشده است. دسترس‌پذیری دامنه، DNS، TLS، نسخه‌ی ابزارهای صوت در تصویر و backup تأیید نشده‌اند. این محدودیت نتیجه‌ی گزارش را مشخص می‌کند: پیکربندی استقرار وجود دارد و قابل تحلیل است، اما آمادگی کامل بهره‌برداری یا موفقیت live deployment ادعا نمی‌شود. آزمون فنی محدود انجام‌شده، جای آزمون عملیاتی کل stack را نمی‌گیرد.

فصل دوازدهم خلاصه نتیجه‌گیری و کارهای آینده

۱۲-۱. تطبیق با هدف‌های مصوب

هسته‌ی ثبت بیمار و داده‌ی بالینی، پیوند اسناد و صوت، Context و timeline، اتصال مدل آماده و گفتگوی ماندگار در کد وجود دارد. قابلیت شواهد، IDهای واقعی پرونده را کنترل می‌کند و داده‌ی مشتق با فایل اولیه جدا نگهداری می‌شود. این مجموعه بخش مهمی از اهداف Backend و AI پروپوزال را پوشش می‌دهد. APIها و migrationها نیز مبنایی برای توسعه و آزمون تکرارپذیر فراهم می‌کنند.

چند هدف در سطح محدودتر اجرا شده‌اند: استخراج سند به متن و تحلیل آزاد می‌رسد، نه فیلد بالینی معتبر؛ Citation به رکورد پرونده محدود است، نه مقاله‌ی پزشکی؛ خلاصه‌ی نمای بیمار ساختاریافته است، نه گزارش روایی مستقل؛ و STT چند ارائه‌دهنده دارد اما کیفیت روی داده‌ی مرجع سنجیده نشده است. ایمنی فعلی AI-first با اصل قطعیِ طرح اولیه تفاوت دارد. بیان این تفاوت‌ها به خواننده اجازه می‌دهد محصول را بر پایه‌ی آنچه دارد ارزیابی کند.

جدول ۷ تطبیق خروجی با هدف‌های طرح

هدف	وضعیت نسخه‌ی موجود	کار باقی‌مانده
هسته‌ی داده و API	موجود با مدل و migration	تقویت تراکنش و سیاست حذف فایل
اسناد و مقادیر منتخب	متن و تحلیل سند موجود	فیلدهای معتبر و provenance جزئی
Context و timeline	موجود و زنده	انتخاب مرتبط و اعلام truncation
مدل آماده و گفتگو	سه adapter متن و گفتگو	ارزیابی معنایی و بازتولید پاسخ
Citation	رکورد پرونده	پشتیبانی معنایی و بازیابی بیرونی
Speech-to-Text	سه مسیر provider	ارزیابی واقعی و metadata نتیجه
Prescription Safety	AI-first و قواعد مستقل	تطبیق ایمنی با طرح و مرجع معتبر
استقرار	Docker و Nginx پیکربندی شده	TLS و اجرای stack و backup
ارزیابی کاربر و بالینی	نتیجه‌ی تأییدشده موجود نیست	پروتکل داده‌ی مرجع و مرور انسانی

۱۲-۲. سهم مهندسی و حدود نتیجه

سهم اصلی حوزه‌ی عرفان در اتصال هسته‌ی داده، API و سرویس‌های هوشمند است. تفکیک قرارداد provider، ساخت Context مشترک، حفظ متن خام، ذخیره‌ی سؤال پیش از تماس AI و کنترل ID شواهد

نمونه‌هایی از تصمیم‌های قابل مشاهده‌اند. این تصمیم‌ها مسیر توسعه و رسیدگی به خطا را روشن‌تر می‌کنند. نتیجه‌ی آن‌ها یک نمونه‌ی نرم‌افزاری قابل بررسی است، نه یک سیستم پزشکی تأییدشده.

رابط بالینی این قابلیت‌ها را در گردش کار پزشک به هم متصل می‌کند. سهم اصلی عرفان در هسته‌ی داده و سرویس‌های هوشمند، سهم اصلی ایمان در رابط و سهم مشترک در معماری و یکپارچه‌سازی طبق پروپوزال تفکیک شده است. قرارداد داده و رفتار قابل مشاهده‌ی کد مبنای شرح این سهم‌هاست.

بازاجرای منتخب در این بازبینی، ۵۹ مورد موفق و دو خطای setup ذخیره‌ساز داشت. جزئیات اجرا در فصل یازدهم ثبت شده است. برای ارزیابی اثر سامانه بر زمان مرور پرونده، خطای درمان یا پیامد بیمار هنوز داده‌ی مرجع و مطالعه‌ی متناسب با آن هدف لازم است. نتیجه‌ی فعلی، ارزیابی محدود قراردادهای نرم‌افزاری است.

۱۲-۳. اولویت‌های مهندسی

اولویت اول بستن شکاف‌های مسیر موجود است: دانلود مجوزدار در رابط، سازگاری سقف صوت با Nginx، نصب ابزارهای صوت در تصویر، اعمال واقعی proxy در STT و پاک‌سازی فایل موقت در تمام شکست‌ها. این کارها به قابلیت‌های تعریف‌شده مربوط‌اند و پیش از اضافه کردن سرویس جدید، قابلیت اعتماد نمونه را بهبود می‌دهند. هر اصلاح باید با تستی همراه باشد که خطای پیشین را واقعاً بازسازی می‌کند.

اولویت دوم مدیریت وضعیت و منشأ است. جدول کار پایدار، شناسه‌ی درخواست، retry کنترل‌شده و نسخه‌ی ورودی برای خروجی مشتق، مشکلات crash و تکرار را بهتر پوشش می‌دهند. برای چت باید provider، model، نسخه‌ی prompt و fingerprint زمینه ثبت شود؛ در صورت نیاز به snapshot، سیاست نگهداری و محرمانگی آن تعیین گردد. اطلاعات source نباید به URL موقتی یا log حساس وابسته بماند.

اولویت سوم امنیت عملیاتی است: TLS، secretهای معتبر، session امن، کنترل نرخ و بازبینی .log حذف بیمار باید به سیاست روشن فایل‌ها و backup متصل شود. مجوزهای سازمانی، مشاهده و ویرایش و ممیزی باید در صورت گسترش به چند مرکز جدا شوند. این اقدامات شرط توسعه‌ی واقعی‌اند، ولی هیچ‌یک به‌تنهایی گواهی انطباق یا تأیید پزشکی ایجاد نمی‌کند.

۱۲-۴. برنامه‌ی ارزیابی و توسعه‌ی علمی

برای اسناد، مجموعه‌ی مصنوعی دارای متن و فیلد مرجع باید ساخته شود و coverage هر صفحه، CER و صحت مقادیر جدا سنجیده شوند. برای صوت، صدای کنترل‌شده‌ی تک‌گوینده و چندگوینده با transcript مرجع و قواعد عادی‌سازی نیاز است. نتیجه‌ی Google و Speechmatics و OpenRouter باید با تنظیم یکسان و همراه اطلاعات provider ثبت شود. آزمون latency نیز با کیفیت متن جمع شود تا راهکار سریع ولی ناقص برتر شمرده نشود.

برای گفتگو، پرسش‌ها باید شامل داده‌ی کافی، داده‌ی غایب، داده‌ی متناقض و متن سند دستورگونه باشند. ارزیاب باید مشخص کند کدام ادعا به کدام رکورد متکی است و آیا شناسه‌ی معتبر واقعاً از جمله حمایت

می‌کند. مقایسه‌ی بدون Context و همراه Context باید روی مجموعه‌ی ثابت و با روش امتیازدهی مشخص انجام شود. وجود prompt ایمنی یا Citation، نتیجه‌ی این آزمایش را از پیش تعیین نمی‌کند.

گسترش بازیابی پزشکی زمانی معنادار است که منبع، نسخه، تاریخ و شرایط استفاده‌ی آن تعیین شده باشد. می‌توان رابطی محدود برای یک منبع رسمی طراحی و کیفیت انتخاب و استناد را مستقل ارزیابی کرد. RAG دائمی یا پایگاه برداری تنها وقتی افزوده شود که ارزش آن نسبت به روش ساده نشان داده شود. در ایمنی دارو نیز بازگرداندن قواعد معتبر یا ترکیب کنترل‌شده‌ی قواعد و توضیح AI باید همراه پوشش و سنجش منفی کاذب باشد.

۱۲-۵. نتیجه‌ی نهایی

Clinical AI Copilot موجود، مسیرهای اصلی ثبت و مرور پرونده و استفاده‌ی کنترل‌شده از ابزارهای آماده را در یک نمونه‌ی تحت وب به هم متصل می‌کند. Backend زمینه و دسترسی و ماندگاری را مدیریت می‌کند و داده‌ی مشتق را از اصل فایل جدا نگه می‌دارد. گزارش حاضر نشان می‌دهد این مبنا قابل توسعه است، اما برای تحلیل بالینی معتبر، بازیابی پزشکی بیرونی و بهره‌برداری عملیاتی به ارزیابی و اصلاحات مشخص دیگری نیاز دارد.

منابع و مراجع

منابع با شماره‌ی یکتا معرفی می‌شوند. ارجاع‌های شماره‌دار مبانی فنی را پشتیبانی می‌کنند؛ مسیر فایل‌ها و نام تابع‌ها شواهد پیاده‌سازی پروژه‌اند. تاریخ دسترسی منابع وب ۱۴ اکتبر ۲۰۲۶ است و شواهد مخزن و نتایج آزمون نیز در فصل‌های مربوط توضیح داده شده‌اند.

[۱] FastAPI. Background Tasks. مستندات رسمی؛ مبنا برای اجرای کار پس از ارسال پاسخ و تفاوت آن با صف مستقل.

https://fastapi.tiangolo.com/tutorial/background-tasks/

[۲] SQLAlchemy. Session Basics، مستندات نسخه‌ی ۲. مبنا برای مدیریت Session و تراکنش؛ انتخاب تنظیمات خاص از کد پروژه برداشت شده است.

https://docs.sqlalchemy.org/en/20/orm/session_basics.html

[۳] Next.js. Self-hosting. مستندات رسمی استقرار برنامه و خروجی runtime.

https://nextjs.org/docs/app/guides/self-hosting

[۴] PostgreSQL. Constraints، مستندات نسخه‌ی ۱۷. مبنا برای قیود یکتایی و خارجی و رفتار حذف.

https://www.postgresql.org/docs/17/ddl-constraints.html

[۵] Google AI for Developers. Gemini API text generation. مرجع قالب تولید متن؛ نام مدل‌های موجود در تنظیمات پروژه، تأیید دسترس‌پذیری روز مدل نیست.

https://ai.google.dev/gemini-api/docs/text-generation

[۶] OpenRouter. API overview. مرجع قرارداد درخواست و پاسخ API؛ نمونه‌ی کد پروژه مستقیماً بررسی شده است.

https://openrouter.ai/docs/api/reference/overview

[۷] Speechmatics. Batch speech-to-text quickstart. مرجع گردش job و دریافت transcript؛ کیفیت فارسی در این گزارش اندازه‌گیری نشده است.

https://docs.speechmatics.com/speech-to-text/batch/quickstart

[۸] pypdf. Extract Text from a PDF. مرجع تفاوت لایه‌ی متن، سند اسکن‌شده و OCR.

https://pypdf.readthedocs.io/en/stable/user/extract-text.html

[۹] Docker. Control startup and shutdown order in Compose. مرجع depends_on و سلامت سرویس؛ وجود پیکربندی با استقرار موفق فرق دارد.

https://docs.docker.com/compose/how-tos/startup-order/

[۱۰] OWASP. HTML5 Security Cheat Sheet. مرجع بحث دسترسی JavaScript به localStorage، خطر XSS و استفاده از cookie دارای HttpOnly.

https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

پیوست الف. مسیرهای منتخب رابط برنامه‌نویسی (API)

این نمونه‌ها مسیرهای اصلی ساخت پرونده، ساخت زمینه، پردازش سند و گفتگوی بالینی را نشان می‌دهند. فهرست کامل مسیرها و قراردادهای OpenAPI در مخزن نگهداری می‌شود و برای پرهیز از تکرار در این گزارش بازنشر نشده است.

جدول الف-۱. مسیرهای منتخب API

مسیر منتخب	کارکرد در گردش کار
POST /api/v۱/patients	ایجاد پرونده‌ی بیمار
GET /api/v۱/patients {patient_id}/context/	ساخت زمینه از رکوردهای پرونده
GET /api/v۱/patients {patient_id}/timeline/	خواندن خط زمانی مشتق‌شده
POST /api/v۱/patients {patient_id}/documents/	بارگذاری سند برای یک بیمار
POST /api/v۱/documents {document_id}/analyze/	تحلیل اختیاری سند ثبت‌شده
POST /api/v۱/patients {patient_id}/chat/	گفتگوی بدون ماندگاری
POST /api/v۱/patients {patient_id}/conversations/	ساخت گفتگوی ماندگار

پیوست ب. تنظیمات منتخب اجرا

این جدول فقط متغیرهایی را گرد می‌آورد که برای انتخاب سرویس، اتصال ذخیره‌ساز یا نشانی رابط اهمیت مستقیم دارند. مقدارهای محرمانه در این گزارش درج نشده‌اند؛ مقدار نهایی باید با فایل محیط و پیکربندی همان استقرار تطبیق داده شود.

جدول ب-۱. متغیرهای منتخب و حدود تفسیر

متغیر	نقش و نکته‌ی پیاده‌سازی
POSTGRES_DB	نام پایگاه داده؛ fixture آزمون نام جداگانه دارد.
MINIO_ENDPOINT	نشانی ذخیره‌ساز؛ باید برای مرورگر قابل دسترسی باشد.
STT_PROVIDER	انتخاب سرویس پیش‌فرض گفتار از تنظیمات.
STT_FALLBACK_PROVIDERS	ترتیب جایگزین‌های گفتار؛ زنجیره با موفقیت یا خطای نهایی متوقف می‌شود.
AI_PROVIDER	انتخاب ارائه‌دهنده‌ی مدل؛ باید با شناسه‌ی مدل و کلید سازگار باشد.
AI_MODEL	شناسه‌ی مدل متنی؛ به‌تنهایی دسترس‌پذیری یا ارزیابی بالینی را ثابت نمی‌کند.
NEXT_PUBLIC_API_URL	نشانی API در رابط وب؛ هنگام build وارد bundle فرانت‌اند می‌شود.

University of Tehran

College of Farabi

Faculty of Engineering

Department of Computer Engineering

Design and Implementation of an AI Clinical Copilot Based on Integration of Patient Context and Validated Medical Knowledge

By

Erfan Karbalai

Supervisor

دکتر کاظم فولادی قلعه

A Bachelor's Project Report in Computer Engineering

Submission month and year: 15 Mehr 1405 (15 October 2026)